

Deep Learning-Based Multi-Object Tracking: A Comprehensive Survey from Foundations to State-of-the-Art

Momir Adžemović ^a

^a*Department of Computer Science, Faculty of Mathematics, University of Belgrade, Studentski trg
16, Belgrade, 105104, Serbia*

Abstract

Multi-object tracking (MOT) is a core task in computer vision that involves detecting objects in video frames and associating them across time. The rise of deep learning has significantly advanced MOT, particularly within the tracking-by-detection paradigm, which remains the dominant approach. Advancements in modern deep learning-based methods accelerated in 2022 with the introduction of ByteTrack for tracking-by-detection and MOTR for end-to-end tracking. Our survey provides an in-depth analysis of deep learning-based MOT methods, systematically categorizing tracking-by-detection approaches into five groups: joint detection and embedding, heuristic-based, motion-based, affinity learning, and offline methods. In addition, we examine end-to-end tracking methods and compare them with existing alternative approaches. We evaluate the performance of recent trackers across multiple benchmarks and specifically assess their generality by comparing results across different domains. Our findings indicate that heuristic-based methods achieve state-of-the-art results on densely populated datasets with linear object motion, while deep learning-based association methods, in both tracking-by-detection and end-to-end approaches, excel in scenarios with complex motion patterns.

Keywords: deep learning, multi-object-tracking, tracking-by-detection, end-to-end tracking

1. Introduction

Multi-object tracking (MOT) is a computer vision task that involves detecting and tracking objects of a given category in a video. It operates without prior knowledge of the objects' appearance or their location. Compared to object detection applied to a video, it additionally requires associating detections across frames by assigning them unique object IDs. Solving MOT results in a set of tracks, where each track consists of all detections belonging to the same object. This task is challenging, especially when aiming for a general solution. The main difficulty arises from frequent occlusions, where MOT algorithms, i.e. trackers, may switch object IDs or lose track of some objects. Additionally, objects with similar appearances make it difficult to distinguish between them, while unpredictable dynamic motion further complicates tracking. Despite these challenges, MOT is essential in various fields that require real-time video analysis. Applications include autonomous driving [13, 49, 64], sports analysis [21, 20], retail [42], robotics [92, 4], biology [101], and surveillance [27, 5], among others. The emergence of deep learning-based methods led to introduction of tracking-by-detection frameworks. This paradigm started with a pioneering work, Simple Online and Real-time Tracker (SORT) [8], demonstrated the effectiveness of using CNN object detectors along with Kalman filtering (KF) for motion prediction and Hungarian matching [63] for detection association across frames. Deep SORT further improved tracking accuracy with the integration of deep learning-based appearance extraction in the association step for more robust tracking in complex scenarios. From that point onward, deep learning is staple in MOT.

Deep learning-based methods in tracking-by-detection paradigm compared to the SORT pioneer algorithm have improved by multiple folds. First, object detection methods have improved by a large margin. New and improved versions of existing architecture families are released [44, 83, 73]. Additionally, novel transformer-based architectures are also now available [16, 106]. Second, association methods have improved in various aspects that include: improved appearance feature extractors [3, 56], integration of camera motion compensation (CMC) methods for camera movement robustness, improved motion models [91, 39, 2], and many association heuristics [103, 15, 94, 75, 52, 93, 43]. Lastly, offline methods based on graph neural networks (GNN) [108] have been introduced to solve the association with higher accuracy compared to online variants.

*Corresponding author

Email address: pd222011@alas.matf.bg.ac.rs (Momir Adžemović )

More paradigms have emerged, capable of performing both detection and association in an end-to-end manner. One of such paradigms are tracking-by-query methods with MOTR [99] and TrackFormer [57] as the pioneers. These methods are based on the DETR [16] end-to-end detection model. Compared to the tracking-by-detection methods, which often require domain knowledge and heuristics for strong performance, end-to-end methods are general solutions that are able to fully learn detection and association directly from data.

With the rapid advancements in deep learning-based MOT, a systematic review is necessary to categorize existing approaches, highlight emerging trends, and identify open research challenges. This paper presents a comprehensive survey of modern deep learning-based algorithms. We specifically examine advancements in both tracking-by-detection and end-to-end paradigms, with an emphasis on state-of-the-art methods. To evaluate their performance across different domains, we compare these methods on multiple benchmarks containing 2D data collected from a single camera. Furthermore, we define a custom benchmark based on existing datasets to evaluate how well each tracking algorithm generalizes. Our contributions are as follows:

- We present a detailed survey of modern deep learning-based methods, with an emphasis on recent works (i.e., from 2022) that have not been covered in detail in existing surveys. The survey is structured into two parts: the first part focuses on the tracking-by-detection paradigm, while the second covers end-to-end tracking methods.
- We identify key categories within the tracking-by-detection paradigm and systematically categorize each reviewed method.
- We compile experimental results from multiple benchmarks to analyze tracker performance across different domains. In addition, we evaluate tracker generality by aggregating and comparing results across benchmarks.
- We provide a comprehensive background, ensuring accessibility for readers with diverse expertise.

2. Related work

Several comprehensive surveys on multi-object tracking have been published in recent years [19, 28, 55, 64, 45, 85, 62, 80]. We present an overview of these surveys along with their topics in Table 1. We group these papers into *general* and *domain* categories. The general category consists of papers that review trackers without focusing on any specific domain or architecture. The domain category provides an overview of trackers applied to a specific domain.

General literature surveys. In [19], the authors review the application of deep learning to different tracker components: object detection, feature extraction, motion prediction, affinity computation, and data association. This survey primarily covers works from 2016 to 2019, with a strong focus on tracking-by-detection approaches, which dominated the field at the time. In [55], the authors examine the transition from traditional to deep learning-based methods, structuring their analysis around fundamental tracker components. In [28], the authors review machine learning-based data association methods, focusing on their connections to linear assignment and analyzing various optimization approaches. In [62], they provide a broad overview of data association techniques, identifying effective methods while discussing challenges and open problems that future research should address. In [85], the authors conduct a detailed review of embedding-based MOT approaches, categorizing different embedding techniques and evaluating their effectiveness for feature representation and association. This survey also summarizes widely used MOT datasets and examines the advantages of state-of-the-art embedding strategies. In [80], the authors review graph-based data association methods in MOT, highlighting the role of graph structures in modeling object dependencies and analyzing recent deep learning implementations alongside traditional approaches.

Domain literature surveys. In [64], the authors review MOT approaches specifically designed for autonomous vehicles, analyzing challenges such as occlusions, varying lighting conditions, and real-time processing constraints. In [45], they conduct a systematic literature review of MOT in traffic environments, covering techniques, datasets, and evaluation metrics while highlighting key challenges and future research directions.

Our focus. Our survey paper falls into the general surveys category. We focus on two main groups of works: pivotal studies that directly influenced modern trackers (e.g., SORT [8] and Deep SORT [89]) and modern state-of-the-art deep learning-based trackers, beginning with tracking-by-detection approaches and extending to end-to-end trackers. For performance comparisons, we consider only high-performance trackers published from 2022 onward. This makes our work complementary to well-known MOT survey papers [19, 55, 85], as they primarily cover older methods. Furthermore, to assess the generalizability of tracking methods across different domains, we define our own benchmark using weighted macro-averaged results across multiple datasets from diverse domains.

Reference	Topic	Group	Year
[19]	Deep learning in MOT: Overview of object detection models, feature extraction, motion prediction, affinity computation, and association methods	General	2019
[28]	Model-based association methods in MOT	General	2020
[64]	MOT methods for autonomous vehicles	Domain	2021
[55]	Transition from traditional to deep learning-based MOT methods: Emphasis on tracking-by-detection approaches	General	2021
[45]	MOT methods in traffic environments	Domain	2022
[62]	Association methods in MOT: Successes and future challenges	General	2022
[85]	Embedding methods in MOT: A comprehensive review	General	2022
[80]	Graph-based data association in MOT	General	2023

Table 1: A summary of literature reviews on Multi-Object Tracking (MOT).

3. Background

We provide the foundational concepts necessary to follow this survey. First, we formally define the multi-object tracking (MOT) problem. Afterwards, we present an overview of the metrics used to evaluate tracker’s performance. Finally, we introduce standard MOT benchmark datasets.

3.1. Problem Formulation

Multi-object tracking is performed on a video that captures a scene containing a set of objects of interest. Let K be a set of identities of objects that are present in the video over a discrete time interval $\{1, 2, \dots, T\}$. At any time t , there exists a set of visible objects:

$$\mathbf{X}_t = \{\mathbf{x}_t^k \mid k \in K_t\} \quad (1)$$

where $\mathbf{x}_t^k$ is the ground truth¹ of an object with identity k at time t , and $K_t \subseteq K$ is a set of visible objects at time t . Objects may enter, exit, or even return to the scene, which leads to K_t varying over time. The objective is to compute the trajectories of all objects in the video. The trajectory of an object with identity k , visible at times T_k , is defined as:

$$\boldsymbol{\tau}^k = \{\mathbf{x}_t^k \mid t \in T_k\} \quad (2)$$

We note that $\boldsymbol{\tau}^k$ can be fragmented due to object being occluded or out of view during some parts of the video. The set of all video ground truth tracks is defined as:

$$\boldsymbol{\tau} = \{\boldsymbol{\tau}^k \mid k \in K\} \quad (3)$$

A MOT tracker is an algorithm that predicts video tracks $\hat{\boldsymbol{\tau}}$, aiming to closely match the ground truth tracks. In the next section, we provide an overview of standard metrics used to evaluate tracker performance by comparing the predicted $\hat{\boldsymbol{\tau}}$ with the ground truth $\boldsymbol{\tau}$. We denote $\hat{\boldsymbol{\tau}}^k$ as the *predicted track* and $\hat{\mathbf{x}}_t^k$ as the *predicted detection* (or just *detection*).

We illustrate the general MOT pipeline in Figure 1. The input to the tracker is a sequence of raw video frames, and the output consists of sets of bounding boxes for each frame with their predicted coordinates, classes, and identities.

3.2. Evaluation Metrics

We describe the metrics used to compare predicted tracks, $\hat{\boldsymbol{\tau}}$, with ground truth tracks, $\boldsymbol{\tau}$. We begin by defining the error types, followed by the similarity measures and matching² between the ground truth and predictions. Finally, we provide an overview of commonly used metrics for evaluating tracking methods.

Error types. We define the following error types:

¹The ground truth annotation of an object, $\mathbf{x}_t^k$, includes annotated features. For 2D bounding box annotations, in addition to the identity, the ground truth provides the horizontal position (x), vertical position (y), width (w), height (h), and class (c).

²We use the term *match* to refer to pairing a prediction with a ground truth during evaluation, and *association* to refer to pairing a track with a detection during inference.



Fig. 1. Illustration of a general MOT pipeline. Frames are taken from scene *DanceTrack0001* [78].

- *False positive track*: A tracked object does not exist.
- *False negative track*: An object is not recognized throughout the video.
- *Identity switch*: Two or more tracks have mixed detection matches.
- *False positive detection*: An object's ground truth does not exist at a specific time (e.g., the object is out of view) but it was still falsely detected.
- *False negative detection*: An object's ground truth exists but is not detected at a specific time.
- *Localization error*: An object's detection offsets from the ground truth.

The first three types of errors are classified as *association errors*, and last three are *detection errors* [53]. In Figure 2, we illustrate an example of correct tracking alongside examples of each previously mentioned error type. Localization errors are not explicitly visualized, as they may occur in all cases, given that predicted detections almost never perfectly match the ground truth.

Similarity measure. In Figure 2, we visually observe that the ground truths and predicted detections match up to some point. To qualitatively evaluate trackers, we first need to define a measure of similarity. The *similarity score* should be chosen based on the domain, with possible values usually ranging from 0 to 1. For a prediction to match a ground truth, a similarity threshold must be met. If the objects' detections are represented as 2D or 3D bounding boxes, the standard IoU (Intersection over Union), also known as Jaccard index, can be used as the similarity measure. Alternatively, if detections are points (e.g., for pose estimation), a similarity measure based on Euclidean distance [53] is more appropriate. In this paper, we focus on videos with objects detected as 2D bounding boxes.

Bijjective Matching. During evaluation, multiple detections can overlap with a single ground truth, and vice versa. Effectively, all possible matching pairs form a bipartite graph, where each prediction belongs to the first group of nodes, and each ground truth belongs to the second group. The edges are weighted based on negative similarity (or positive distance) scores for minimal cost matching. A score threshold is often applied to determine whether two nodes can be matched; otherwise, the edge weight is treated as infinite. This setup represents a standard bipartite graph optimal assignment problem, where the objective is to maximize similarity or a metric of interest. It is commonly solved using the Hungarian algorithm [63]. Matching is generally performed either at the detection level (i.e., per frame) or at the

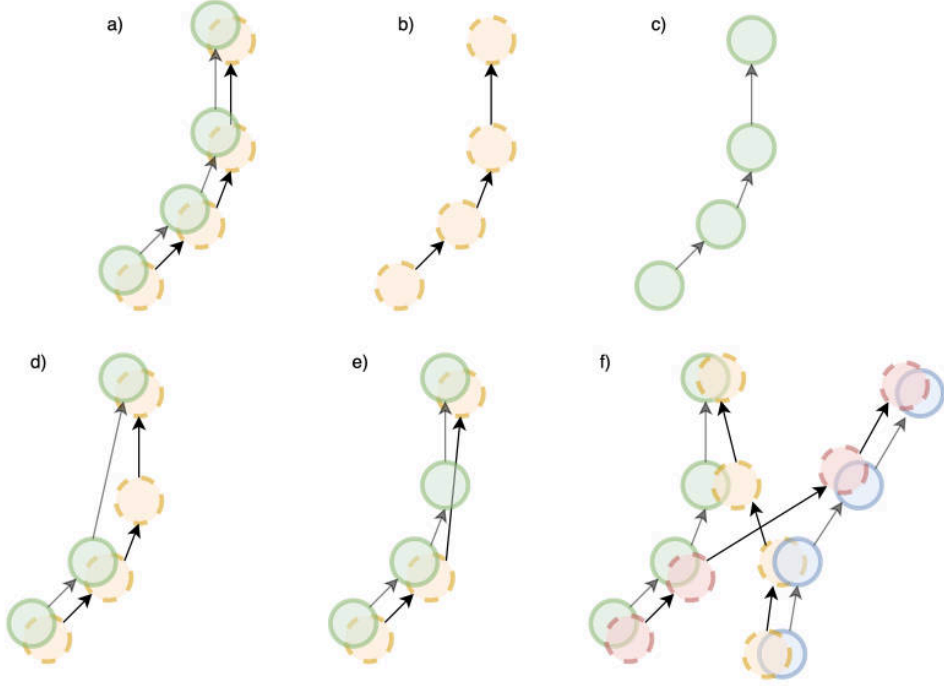


Fig. 2. The figure illustrates six cases of tracking results. We differentiate between ground truth tracks, shown with ground truths as solid circles, and predicted tracks, shown with detections as dashed circles. In case (a), the predicted and ground truth tracks match at every moment. In case (b), we observe only the predicted track (false positive track). Similarly, in case (c), we see only the ground truth track, as the predicted track is missing (false negative track). In case (d), the predicted track misses the third detection (false negative detection), and (e) contains an extra, third detection (false positive detection). Finally, in case (f), both predicted tracks initially align but switch identities at the third step (identity switch). In all cases, we observe that the ground truths and predictions do not align perfectly, leading to localization errors.

track level (i.e., across the video). In further text, we cover standard MOT metrics used to compare models on benchmarks.

MOTA and MOTP metrics. One of the earliest multi-object tracking metrics, which is still widely used today, is MOTA (Multi-Object Tracking Accuracy) along with MOTP (Multi-Object Tracking Precision) [7, 53]. In this case, bijective matching is performed at the detection level, with default IoU threshold equal to 0.5. Specifically, the MOTA metric is defined as follows:

$$\text{MOTA} = 1 - \frac{|\text{FP}| + |\text{FN}| + |\text{IDSW}|}{|\text{gtDet}|} \quad (4)$$

where $|\text{FP}|$ is the total number of false positive detections, $|\text{FN}|$ is the total number of false negative detections, $|\text{IDSW}|$ is the total number of identity switches, and $|\text{gtDet}|$ is the total number of ground truth detections. The MOTA metric values range from $-\infty$ to 1, where higher values indicate better performance.

Since MOTA does not account for localization error, which measures average similarity over correct matches, MOTP is introduced and defined as:

$$\text{MOTP} = \frac{1}{|\text{TP}|} \sum_{m \in \text{TP}} S_m \quad (5)$$

where $|\text{TP}|$ is the total number of true positives (correct matches), and S_m is the similarity score of the matched pair m . A higher MOTP indicates a higher average similarity score between matched ground truths and predictions. Matching is performed at the detection level by optimizing both MOTA and MOTP.

While MOTA and MOTP are useful metrics, relying solely on them is insufficient. Firstly, MOTA is more influenced by detection performance than association performance [7, 53]. Additionally, it does not account for when an identity switch occurs—whether at the start, middle, or end of a track. For these reasons, other identification metrics are often used.

IDF1 metric. Opposite of the MOTA, the IDF1 metric calculates bijective matching at track level, which results in a different type of false positive and false negative detections. The IDF1 metric is defined as follows [7, 53]:

$$\text{IDF1} = \frac{|\text{IDTP}|}{|\text{IDTP}| + 0.5 \cdot |\text{IDFN}| + 0.5 \cdot |\text{IDFP}|} \quad (6)$$

where $|\text{IDTP}|$ represents the number of true positive identity matches, $|\text{IDFN}|$ denotes the number of false negative identity matches, and $|\text{IDFP}|$ denotes the number of false positive identity matches. Matching is performed at the track level by optimizing the IDF1 metric. In contrast to the MOTA metric, the IDF1 metric is biased toward association performance and is sensitive to identity switches. To further evaluate tracking performance, ID-Recall (IDR) and ID-Precision (IDP) are also used:

$$\text{ID-Recall} = \frac{|\text{IDTP}|}{|\text{IDTP}| + |\text{IDFN}|} \quad (7)$$

$$\text{ID-Precision} = \frac{|\text{IDTP}|}{|\text{IDTP}| + |\text{IDFP}|} \quad (8)$$

where ID-Recall measures the percentage of true positive identity matches relative to the total number of ground truth matches (true positives and false negatives). ID-Precision measures the percentage of true positive identity matches relative to the total number of predicted matches (true positives and false positives). The IDF1 metric combines both IDP and IDR into a single metric, providing a balanced evaluation of identity matching performance [71].

HOTA metric. HOTA (Higher Order Tracking Accuracy) is a metric designed to evaluate MOT performance by jointly considering detection, association, and localization quality. HOTA measures how well the tracks of matched detections align and averages this over all matched detections [53]. Unlike earlier metrics such as MOTA and IDF1, which emphasize either detection or association, HOTA balances these aspects by explicitly separating detection and association errors while incorporating spatial and temporal consistency. For this metric, matching is performed at the detection level, ensuring evaluation is based on individual object detections rather than higher-level track groupings.

The HOTA_α metric with a IoU similarity threshold α is defined as the geometric mean of detection DetA_α and association AssA_α metrics:

$$\text{HOTA}_\alpha = \sqrt{\text{DetA}_\alpha \cdot \text{AssA}_\alpha} \quad (9)$$

The DetA_α metric is simply the Jaccard index, a measure of similarity between two sets — ground truths and predictions. It is defined as:

$$\text{DetA}_\alpha = \frac{|\text{TP}|}{|\text{TP}| + |\text{FN}| + |\text{FP}|} \quad (10)$$

where TP is the set of matches between ground truths and predictions, FP is the set of predictions without a corresponding ground truth, and FN is the set of ground truths without a corresponding prediction. The AssA metric is defined as:

$$\text{AssA}_\alpha = \frac{1}{|\text{TP}|} \sum_{c \in \{\text{TP}\}} A(c) \quad (11)$$

where

$$A(c) = \frac{|\text{TPA}(c)|}{|\text{TPA}(c)| + |\text{FNA}(c)| + |\text{FPA}(c)|} \quad (12)$$

In this case, c is a pair of ground truth and prediction IDs. Specifically, set $\text{TPA}(c)$ is the set of true positives (TPs) that have the same ground truth and prediction IDs. Set $\text{FNA}(c)$ is the union of the set of all TPs that have the same ground truth ID as c but a different prediction ID, and the set of all unmatched ground truths with the same ground truth ID as c . Set $\text{FPA}(c)$ is the union of the set of all TPs that have the same prediction ID as c but a different ground truth ID, and the set of all unmatched predictions with the same ID as c [53]. This is best understood through an example. Assume that we have the following set of matched pairs: $\{(a, 1)_1, (a, 1)_2, (b, 2)_2, (a, 1)_3, (b, 2)_3, (a, -)_4, (b, 1)_4\}$, where $(x, y)_t$ represents matched ground truth x with prediction y at time t . A visualization of this example can be seen in Figure 3.

In this case:

- $\text{TPA}((a, 1)_1) = \{(a, 1)_1, (a, 1)_2, (a, 1)_3\}$,
- $\text{FNA}((a, 1)_1) = \{(a, -)_4\}$,
- $\text{FPA}((a, 1)_1) = \{(b, 1)_4\}$,
- hence, $A((a, 1)_1) = \frac{3}{3+1+1} = 0.6$,
- similarly, $A((b, 2)_2) = \frac{2}{2+0+1} = 0.67$

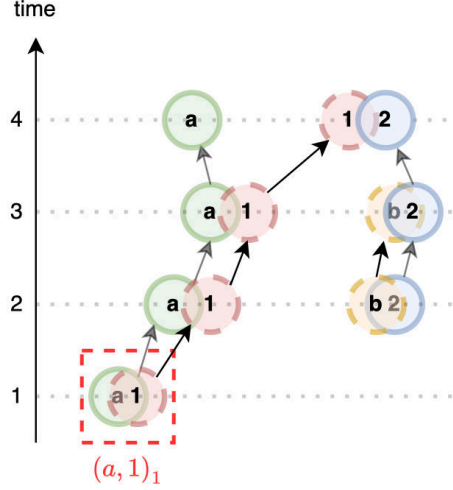


Fig. 3. Example of ground truth and prediction track matching. The TP pair of interest is marked with a dashed red triangle.

Finally, in order to measure localization, HOTA is calculated with different thresholds (0.05 to 0.95 in 0.05 intervals):

$$\text{HOTA} = \frac{1}{19} \sum_{\alpha} \text{HOTA}_{\alpha} \quad (13)$$

Because HOTA considers both types of errors—association and detection—it is often used as the main metric for tracker benchmarks. Besides HOTA, for more detailed comparisons, AssA, DetA, MOTA, IDF1, and IDSW (total number of ID switches) are also evaluated [22, 78, 21, 49]. For all metrics except IDSW, higher values indicate better tracker performance.

3.3. Datasets

We provide an overview of the datasets commonly used as standard benchmarks for MOT methods. These datasets will later be used for method comparisons.

MOTChallenge—MOT17 and MOT20. The MOT17 dataset³ consists of 14 short video sequences captured with non-static cameras, predominantly featuring pedestrians with linear motion. These sequences are evenly split between training and test sets. Similarly, the MOT20 dataset includes eight heavily crowded scenes recorded with static cameras, also evenly divided between training and test sets. However, MOT20 contains approximately three times more bounding boxes in total compared to MOT17. The primary challenges in the MOTChallenge benchmark’s scenes are crowded environments and frequent occlusions [22].

DanceTrack. DanceTrack⁴ is a dataset comprising 100 videos of people dancing, each meticulously annotated for the purpose of MOT tracker design and evaluation. This task is particularly challenging due to the dancers’ similar appearances, frequent occlusions, and their rapid, non-linear movements. The dataset is segmented into 40 training videos, 25 validation videos, and 35 test videos. The dataset is highly diverse as it includes multiple genres of dancing: classical, street, pop, large group and sports [78].

SportsMOT. SportsMOT⁵ is a comprehensive multi-object tracking dataset that includes 240 video sequences, totaling over 150,000 frames. The dataset encompasses three categories of sports: basketball, volleyball, and football. It is particularly challenging for object tracking due to the fast and varying speeds of motion and the similar appearances of the objects involved. The dataset is organized into training, validation, and test sets, which contain 45, 45, and 150 video sequences, respectively. Annotations include 2D bounding boxes, player identities, and team labels, enabling evaluations of trackers in complex scenarios involving frequent occlusions, rapid movements, and team-based associations [21].

Datasets overview. Table 2 presents key statistics for five multi-object tracking datasets. MOT17 and MOT20 emphasize dense pedestrian tracking, with MOT20 containing significantly more bounding boxes (1.65M vs. 292K) and tracks (3456 vs. 1342) for the same total video length. On average, MOT20 has nearly five times as many pedestrians per frame as MOT17. When comparing datasets regarding the

³MOTChallenge website: <https://motchallenge.net/>.

⁴DanceTrack GitHub page: <https://github.com/DanceTrack/DanceTrack>.

⁵SportsMOT GitHub page: <https://github.com/MCG-NJU/SportsMOT>.

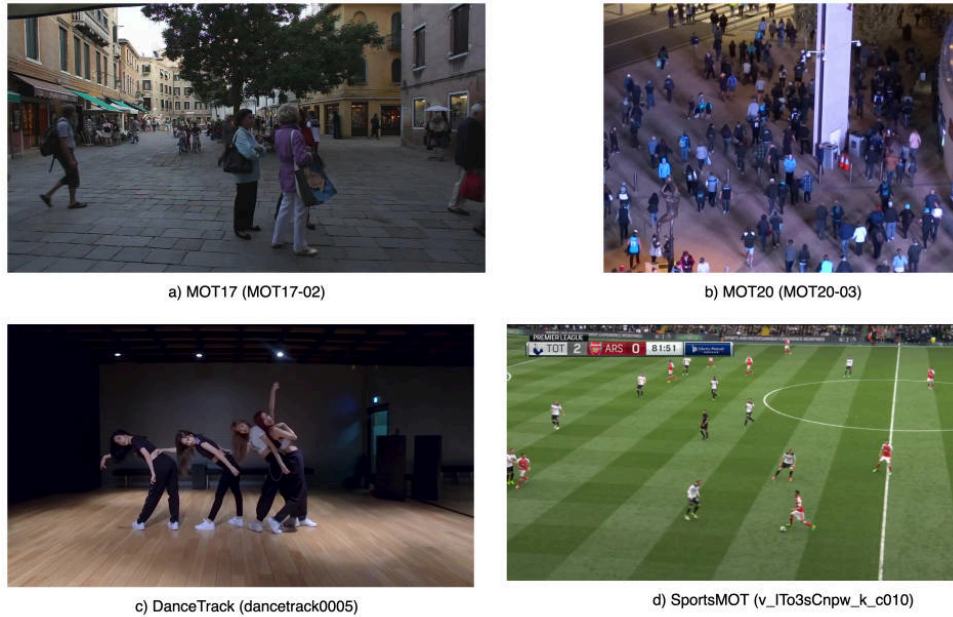


Fig. 4. Sample frames from MOT datasets across different domains: (a) MOT17, (b) MOT20, (c) DanceTrack, (d) SportsMOT.

Dataset	Scenes	Frames	Length (s)	BBox	Tracks
MOT17	14	14235	463	292733	1342
MOT20	8	13410	535	1652040	3456
DanceTrack	100	105855	5292	-	990
SportsMOT	240	150379	6015	1629490	3401

Table 2: Summary of dataset statistics, including the number of scenes, frames, sequence length (in seconds), bounding boxes (BBox), and object tracks. Sources: [22, 78, 21].

video length, DanceTrack and SportsMOT surpass MOTChallenge by an order of magnitude, featuring 105K and 150K frames, respectively, compared to just 27K frames for MOT17 and MOT20. However, we observe that these datasets are much less crowded than MOT20.

As it can be seen from frame samples in Figure 4, these datasets collectively cover various challenges, including dense crowds, rapid motion, and long-term tracking. Evaluating a tracker on a single dataset is not ideal, as it fails to demonstrate generalization across different scenarios and motion patterns. MOTChallenge has objects with predictable motion with very crowded scenes. DanceTrack emphasizes diverse motion patterns and similar appearances. SportsMOT is a more extreme variant, featuring greater motion diversity and higher appearance similarity.

4. Tracking-by-Detection Paradigm

Tracking-by-detection is a simple yet highly efficient paradigm in MOT, excelling in both accuracy and speed. The detection and association are decoupled and performed in independent steps. First, an object detection model detects objects (usually as 2D bounding boxes) in each frame. Then, an association method is used to associate objects across frames. This paradigm is widely adopted due to its modularity, which enables independent optimization of detection and association modules, its flexibility to integrate both traditional algorithms and deep learning models, and its strong performance in modern applications.

To make this section accessible, we begin by reviewing the literature on deep-learning-based object detection models. We then examine online tracking-by-detection methods, starting with the baselines—SORT and Deep SORT. Next, we present clustered MOT methods, covering joint detection and embedding, heuristic-based, motion-based, affinity learning, and offline (graph-based) approaches. Finally, we discuss the strengths, limitations, and future research directions of the tracking-by-detection paradigm. While our focus is on modern deep-learning-based methods, we also acknowledge a few older but highly influential MOT approaches.

4.1. Deep learning object detection models

Object detection models can be roughly categorized into three families: proposal-based methods, grid-based methods, and query-based methods [9]. We briefly introduce each of these families in further text.

Proposal-based object detectors. Proposal-based approaches follow a two-stage pipeline: first, region proposals are generated; then, each proposed region undergoes bounding box localization and classification. One of the earliest methods, R-CNN [36], generates region proposals (i.e., crops) using the selective search algorithm [81] and then applies a convolutional neural network (CNN) [59, 38] classifier for each proposal. Although R-CNN achieves high detection accuracy compared to other methods at the time, its main drawback is slow computation speed. Fast R-CNN [35] improves the efficiency of R-CNN by sharing convolutional features across proposals. First it uses a single CNN pass to obtain image features. Second, it introduces ROI (Region Of Interest) pooling, which pools image features for each proposal obtained through the selective search algorithm. This modification to the R-CNN results in a processing speedup, as the CNN extractor is applied once to the whole image instead of being applied separately to each crop. Fast R-CNN also introduces anchors—predefined bounding boxes of various sizes and aspect ratios—to improve detection accuracy by serving as reference points for object localization [35]. Further improvements were introduced in Faster R-CNN [68], which replaces selective search with CNN-based region proposal networks (RPNs), enabling end-to-end training and significantly improving speed. Mask R-CNN [41] extends Faster R-CNN by adding a branch for pixel-wise instance segmentation, making it suitable for tasks requiring object masks in addition to bounding boxes. Both Faster R-CNN and Mask R-CNN are supported by the Detectron2 framework [37, 90], a modular and extensible library commonly used to implement, train, and evaluate proposal-based object detectors.

The performance of these methods during inference often depends on Non-Maximum Suppression (NMS) [36] to remove duplicate detections and retain the most confident predictions. This results in a two-step inference process, where object detection first outputs raw bounding boxes, potentially containing duplicates, and NMS is subsequently applied to eliminate most of them. NMS requires hyperparameter manual tuning based on the domain to avoid removing valid detections as duplicates.

Grid-based object detectors. YOLO (You Only Look Once) [67, 79] models are state-of-the-art in real-time performance [83] with a single-shot design that directly predicts bounding boxes and class probabilities from an image. This object detection model uses a grid-based approach to detect objects in a single pass, dividing the image into a grid where each cell is responsible for detecting one or more objects. In this case, the object detection problem is formulated as a regression task, optimized using the well-known YOLO loss [67]. The main drawback of the original YOLO, compared to concurrent proposal-based methods, was its difficulty in detecting small objects due to the design of its regression loss. YOLOv2 [65] introduces a bag of tricks to improve performance in terms of both accuracy (e.g. better detection of smaller objects) and speed. Inspired by Fast R-CNN, it introduces anchors into the YOLO family, improving localization accuracy while maintaining real-time performance. YOLOv3 [66] enhances multi-scale predictions using feature pyramid networks (FPNs) [50], further improving robustness for small objects. YOLOv4 [10] introduces an extended bag of tricks, including CSPDarkNet [84] as a backbone and mosaic augmentation, achieving strong performance while being lightweight enough to train on a single NVIDIA 2080 Ti GPU—unlike its predecessors. Ultralytics’ YOLOv5, YOLOv8 and YOLOv11 [46], further optimize the architecture for speed and accuracy, incorporating advanced techniques such as mosaic augmentation, auto-anchor optimization, and better backbone designs. YOLOv5 is the first YOLO model implemented in Python, specifically using PyTorch [60]. YOLOX [34] adopts an anchor-free approach with decoupled heads for classification and regression, achieving strong performance in accuracy and tracking applications [78, 21, 103, 43]. YOLOX is often used for MOT benchmarks [103, 78, 21]. Ultralytics’ YOLO models also tend to be used due to their speed and ease of integration [44, 73, 95]. YOLO models also depend on NMS to remove duplicate detections. Although anchorless versions are less reliant on NMS, they still require its use for optimal performance [34, 46].

Query-based object detectors. DETR (DEtection TRansformer) [16] reformulates object detection as a set prediction problem, eliminating the need for hand-crafted anchors and post-processing steps such as the NMS, enabling both end-to-end training and inference. The model consists of four main components: a CNN backbone (typically ResNet [40]) that extracts image features, an transformer [82] encoder that further enriches these image features using self-attention, a decoder that predicts object embeddings based on learnable queries (learnable positional encodings) through cross-attention with image features, and a prediction head that maps these object embeddings to bounding box coordinates and class labels. DETR’s training process leverages a bipartite matching strategy to assign ground truth objects to predictions. Using the Hungarian algorithm, the model ensures a unique one-to-one assignment, simplifying the pipeline and enabling it to learn to avoid duplicate predictions—thus eliminating the need for NMS. Compared to the proposal-based and YOLO families, DETR is fully end-to-end, as it requires

no NMS postprocessing, which can significantly impact CPU or GPU compute [106]. However, despite these advantages, DETR suffers from slow convergence, particularly for small objects.

DAB-DETR [51] improves upon DETR by introducing dynamic anchor boxes, addressing convergence speed and small object detection issues. DINO [100] builds on DETR with advanced query initialization and contrastive denoising, achieving state-of-the-art results in detection accuracy and robustness. However, DETR models are in general slow compared to YOLO models, unless optimized variants for real-time are used, like RT-DETR [106]. Besides object detection, DETR enabled a family of transformer-based end-to-end tracking methods which we will go through in detail in next section [99, 104, 97, 14, 31, 32].

4.2. SORT

SORT (Simple online and real-time tracking) [8] is the pioneer of tracking-by-detection paradigm. It’s a simple algorithm with three key components: object detection model, association method and track handling. A motion model or an appearance model is often used to improve association performance.

Object detection model. For object detection, original SORT uses Faster R-CNN. This model was state-of-the-art at the time [68, 8]. However, SORT and other tracking-by-detection methods are not tied to a specific object detection model. Faster R-CNN can be easily replaced with any modern, more efficient model (e.g. YOLOX) [103].

Tracker only considers bounding boxes with high detection confidence. An important hyperparameter for tracker performance is the *detection score threshold*, det_τ , which balances false positives and false negatives. A higher threshold reduces false positives but increases false negatives, and vice versa. SORT’s performance is highly sensitive to the choice of det_τ [103].

Motion model. SORT uses a Kalman Filter (KF) estimation model to perform both bounding box motion prediction and noise filtering. A brief KF introduction can be found in Appendix A. Particularly, SORT employs linear constant velocity model for track’s motion prediction. This motion prediction is used during the association step to associate the track with detections. After a successful association, Bayesian inference is applied to combine information from both sources—the motion prediction and the associated detection—to update the track’s state. The KF’s state is modeled as $[x, y, s, r, \dot{x}, \dot{y}, \dot{s}]$, where x and y represent the bounding box center coordinates, s is the bounding box scale (area), r is the bounding box aspect ratio between height and width, $\dot{x}$ and $\dot{y}$ are the velocities of the center coordinates, and $\dot{s}$ is the scale velocity. Notably, this model assumes that the bounding box aspect ratio does not change over time. In context of Bayesian inference, the motion model prediction acts as the *prior*, while the detection serves as the *measurement likelihood* mean. Since object detectors typically do not provide uncertainty estimates for individual coordinates, heuristics are often used to approximate the measurement likelihood variance [8, 89, 3].

Association method. For each frame, SORT uses Hungarian algorithm to solve linear assignment problem in assigning detections to tracks. Association cost between a track’s prediction bounding box and a detection bounding box is equal to negative IoU (Intersection Over Union). If the IoU similarity is below threshold IoU_{min} , the track cannot be associated with the corresponding detection, as they are too far apart. This type of constraints in assignment problems are often referred to as *gates*. The input to a linear assignment solver can be represented as an association cost matrix, where rows correspond to tracks and columns correspond to detections. Linear assignment solver returns three sets as outputs: a set of associated tracks and detections pairs, a set of un-associated tracks, and a set of un-associated detections. Following the association step, the *track handling* logic is responsible for managing all these sets.

Track handling logic. Tracks’ creation and deletion should follow objects that are entering, leaving, and possibly returning to the scene over time. All un-associated detections are considered as potential new tracks with new identities. Creating a new track for each new detection would result in a high false positive ratio, for each object detection false positive. Instead, each track needs to undergo a probationary period, where it needs to be associated with a detection few frames in a row. If at least one frame is missed during the probationary period, the potential new track is deleted. The default probationary period in SORT is 3 frames [8]. Existing tracks are terminated if they are not associated with any detection for T_{lost} consecutive frames. Remembering the tracks is important in order to handle detector false negatives and objects’ occlusions. SORT uses $T_{lost} = 1$ as a default configuration. This means that, with its default configuration, SORT does not handle long-range occlusions.

Algorithm. We define the following track states:

- *active* — A state for objects that are currently being tracked with a successful association in the previous frame.
- *lost* — A state for objects that are currently being tracked but failed to associate in one or more of the most recent frames (e.g. due to occlusion). Once a lost track is successfully associated with a detection, it becomes active again.

- *deleted* — A state for objects that are no longer being tracked and are considered completely lost.
- *new* — A state for objects in a probationary period (potential tracks). Once the probationary period ends, the track becomes active.

These states are important in order to understand the implementation of SORT’s Algorithm 1. The three key steps of the algorithm are noted as: object detection inference, association and track handling. The motion model inference can be considered part of the association process. We omitted the probationary period logic for new tracks to keep the algorithm simpler. For the probationary period, some changes are required in the track handling logic. Specifically, these tracks have to be handled during the *update associated tracks* and *update un-associated tracks* steps.

Algorithm 1 SORT’s pseudocode

Models: object detector Det , Kalman Filter KF ;
Hyper-parameters: association threshold IoU_{min} , detection threshold det_τ , maximum track lost time T_{lost} ;
Other: Hungarian Algorithm HG returns associated pairs, and un-associated singles;
Input: a video sequence V ;
Output: predicted tracks $\hat{\tau}$ of the video.

```

1: Initialization:  $\hat{\tau} = []$ 
2: for frame  $(t, f_t)$  in  $V$  do
3:   # Object detection inference
4:    $D = Det(f_t, det_\tau)$ 
5:
6:   # Association
7:    $\hat{T} = []$  ▷ Tracks’ prior predictions
8:   for  $\tau_k$  in  $get\_alive(\hat{\tau})$  do ▷ alive — active or lost
9:      $\hat{T}.append(KF.predict(\tau_k, t))$ 
10:  end for
11:   $C = -IoU(D, \hat{T}, IoU_{min})$ 
12:   $A, T_{unassoc}, D_{unassoc} = HG(C)$  ▷ A — Associated pairs
13:
14:  # Track handling
15:  for  $(i_t, i_d)$  in  $A$  do ▷ Update associated tracks
16:     $\hat{\tau}^{k_{i_t}} = \hat{\tau}[i_t]$ 
17:     $\hat{\tau}^{k_{i_t}}.update(KF.update(\hat{T}[i_t], D[i_d]))$ 
18:     $\hat{\tau}^{k_{i_t}}.lost = 0$ 
19:  end for
20:  for  $i_t$  in  $T_{unassoc}$  do. ▷ Update un-associated tracks
21:     $\hat{\tau}^{k_{i_t}} = \hat{\tau}[i_t]$ 
22:     $\hat{\tau}^{k_{i_t}}.lost = \hat{\tau}^{k_{i_t}}.lost + 1$ 
23:    if  $\hat{\tau}^{k_{i_t}}.lost \geq T_{lost}$  then
24:       $\hat{\tau}.delete(k_{i_t})$ 
25:    end if
26:  end for
27:  for  $i_d$  in  $D_{unassoc}$  do. ▷ Create new tracks
28:     $d^{i_t} = d[i_d]$ 
29:     $\hat{\tau}.create(d^{i_t})$ 
30:  end for
31: end for

```

4.3. Deep SORT

SORT generally exhibits a high frequency of identity switches [8, 89], primarily due to the low precision of its motion model. This issue is especially pronounced during long occlusions, where prediction errors accumulate. Since SORT’s motion-based association is insufficient in the general case, Deep SORT [89] proposed to combine the motion and appearance information for more accurate associations. Appearance information is extracted from images based on objects’ bounding box crops (pathes). This is achieved using a convolutional neural network (CNN) [38] trained on a re-identification dataset in classification mode to discriminate between objects’ identities [107]. Specifically, a trained CNN extracts each detected object’s appearance embedding vector, which then passes through a softmax classification head to output the

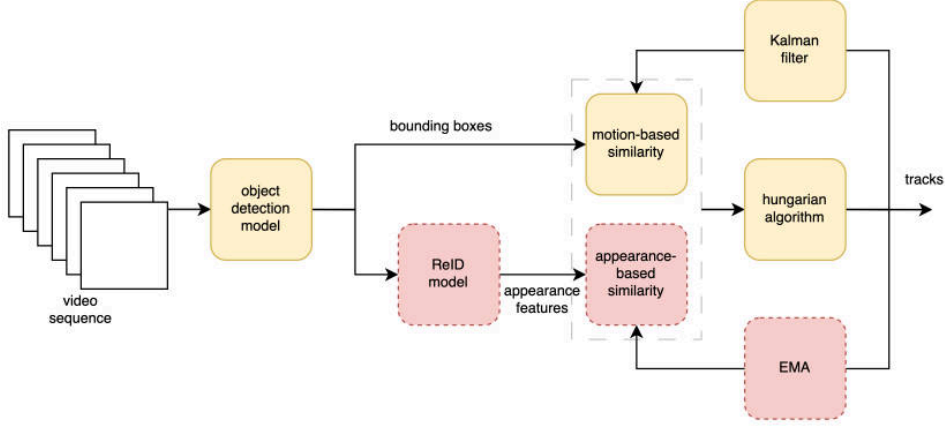


Fig. 5. Overview of the DeepSORT tracking pipeline. Components shown in yellow with solid borders represent the original SORT framework, while components in red with dashed borders denote DeepSORT extensions. An object detection model generates bounding boxes, which are passed through a ReID model to extract appearance features. Similarity is then computed between the appearance features of the detections and those of existing tracks, maintained using a buffered history.

object’s identity. During inference, classification head is dropped and cosine distance is used to compare feature embedding vectors, yielding a similarity score between 0 and 1. A model used for appearance embedding extraction is often called a *re-ID* model, as it is specifically trained for this purpose.

DeepSORT’s high-level tracking pipeline is shown in Figure 5. The detector and the re-ID model operate sequentially. First, the detector outputs a list of bounding boxes. Then, for each bounding box, the re-ID model extracts an appearance embedding, which is used for appearance-based association. Specifically, each track maintains a buffer of appearance feature embeddings. The similarity between a track and a detection is computed as the nearest neighbor distance between the detection’s embedding and any embedding in the track’s buffer.

$$d_A(i, j, \mathbf{E}^D, \mathbf{E}^T) = \min_{\mathbf{E}_{j,k}^T \in \mathbf{E}_j^T} d_{\cos}(\mathbf{E}_i^D, \mathbf{E}_{j,k}^T) \quad (14)$$

where $d_{\cos}$ is the cosine distance, i is the index of the i th detection, j is the index of the j th track, $\mathbf{E}^D$ represents the detection appearance embeddings, and $\mathbf{E}^T$ represents the buffered track appearance embeddings, in which case $\mathbf{E}_{j,k}^T$ is the k th embedding in the j th track’s embedding buffer.

In addition to integrating appearance information into association, Deep SORT introduces several other modifications. First, for the KF state it uses height and width along with their velocity states instead of scale and aspect ratio. This change provides greater flexibility to the KF model, as height and width have decoupled velocities. Second, the cost function for the assignment problem is modified. Instead of relying on the negative IoU, Deep SORT employs the Mahalanobis distance, which accounts for the uncertainty provided by the KF’s predictions. The cost function is then defined as:

$$C_{\text{Deep SORT}}(i, j, \mathbf{D}, \mathbf{T}, \mathbf{E}^D, \mathbf{E}^T) = \lambda d_M(\mathbf{D}_i, \hat{\mathbf{T}}_j) + (1 - \lambda) d_A(i, j, \mathbf{E}^D, \mathbf{E}^T), \quad (15)$$

where $\mathbf{D}_i$ represents the i th detection with corresponding appearance embedding $\mathbf{E}_i$, $\hat{\mathbf{T}}_j$ represents the j th predicted track bounding box, d_M is the Mahalanobis distance, and d_A is the appearance distance. The weight λ balances motion-based and appearance-based association costs. If $\lambda = 1$, the association cost is determined solely by motion and geometric features, whereas if $\lambda = 0$, it is based only on appearance features. Deep SORT uses $\lambda = 0$ as its default configuration [89]. However, motion and geometric features are still used in the gate function calculation, which is defined as follows:

$$G_{\text{Deep SORT}}(i, j, \mathbf{D}, \mathbf{T}, \mathbf{E}^D, \mathbf{E}^T) = \mathbf{1}[d_M(\mathbf{D}_i, \hat{\mathbf{T}}_j) < t^{(M)}] \cdot \mathbf{1}[d_A(i, j, \mathbf{E}^D, \mathbf{E}^T) < t^{(A)}] \quad (16)$$

where $t^{(M)}$ is the Mahalanobis distance threshold and $t^{(A)}$ is the appearance distance threshold. Association is possible only if both motion and appearance conditions are met.

It is worth noting that the Mahalanobis distance is not a popular choice in the modern MOT algorithms due to its limitations [103, 15, 3, 52, 43, 56]. The problem lies in the fact that predictions for lost tracks accumulate uncertainty, which is natural but makes lost tracks cheaper to associate with compared to active tracks. To address this, Deep SORT introduced cascaded association based on track’s time lost which partly resolves this issues. Empirical evidence suggests that this modification becomes less effective as the tracker performance improves [26].

The final significant modification is the decoupled association between active and new tracks, where detections are first associated with active tracks. This prioritization of already tracked objects over

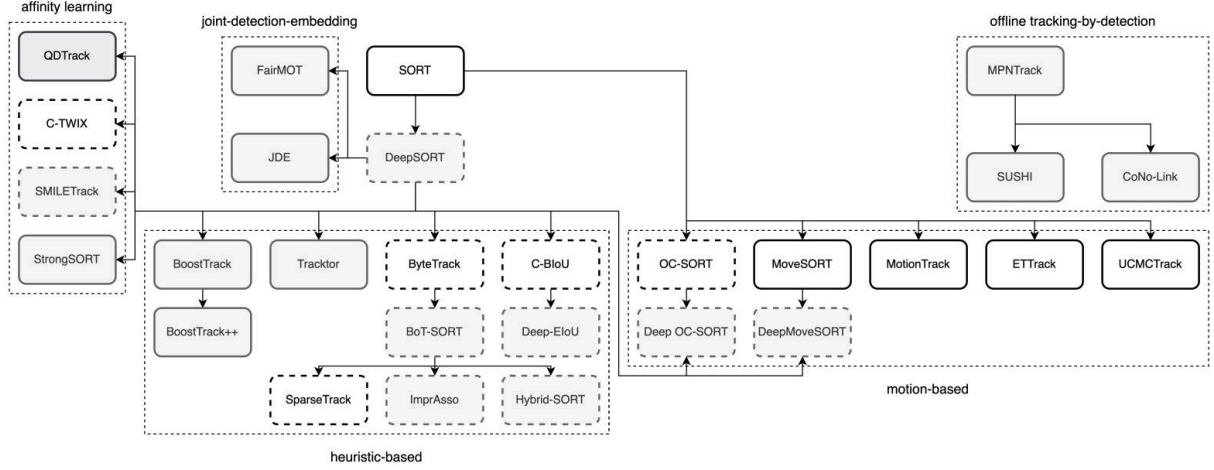


Fig. 6. Evolution of influential tracking-by-detection Methods in MOT. Trackers with shaded boxes employ re-ID model, while trackers with dashed box borders use cascaded association (e.g. Byte).

potential false positives further improves robustness. This modification is applied to modern MOT methods [103, 3, 52, 75, 15, 56, 43].

In summary, the pipeline of Deep SORT, with its improved association performance, differs slightly from that of SORT. It introduces an additional appearance extraction step between detection inference and association, which can also be viewed as a sub-step of the association process. The pipeline remains decoupled, allowing the re-ID model to be improved or replaced without modifying any other component of the tracker [89].

4.4. Tracking-by-detection method categorization

In the following text, we highlight the most important tracking-by-detection methods. Notably, every tracking-by-detection algorithm in MOT can be regarded as a variant of SORT. A graph illustrating the most influential work with emphasis on the recent studies is presented in Figure 6. These algorithms are roughly categorized into the following categories:

- *Joint-detection-embedding* (JDE) methods, which propose a single model that performs both detection and appearance extraction, instead of using separate detection and re-ID models like in Deep SORT.
- *Heuristic-based* methods, which rely solely on association heuristics without introducing any model-related improvements.
- *Motion-based* methods, which enhance association accuracy by directly improving the motion model component.
- *Affinity learning* methods, which learn object affinities based on geometric, motion, or appearance features.
- *Offline* methods, which perform association at a global level after all video detections have been obtained.

It is important to note that most tracking-by-detection algorithms incorporate some form of heuristic to improve association accuracy. However, we refer to heuristic-based methods as those that do not propose any significant improvements (e.g., model enhancements) beyond a set of heuristics for improved association. In the following sections, we briefly review each of these algorithm categories, highlighting their progress over their predecessors.

4.5. Joint detection and embedding tracking methods

JDE. JDE (Joint Detection and Embedding) [87] integrates detection and re-identification (re-id) tasks into a single neural network through joint task learning. The primary motivation behind this approach is to mitigate the slower inference of re-ID models in crowded scenes, where computational requirements scale with the number of cropped regions. By integrating the re-ID model within the detector, both tasks can be performed in a *one-shot* manner, as shown in Figure 7.

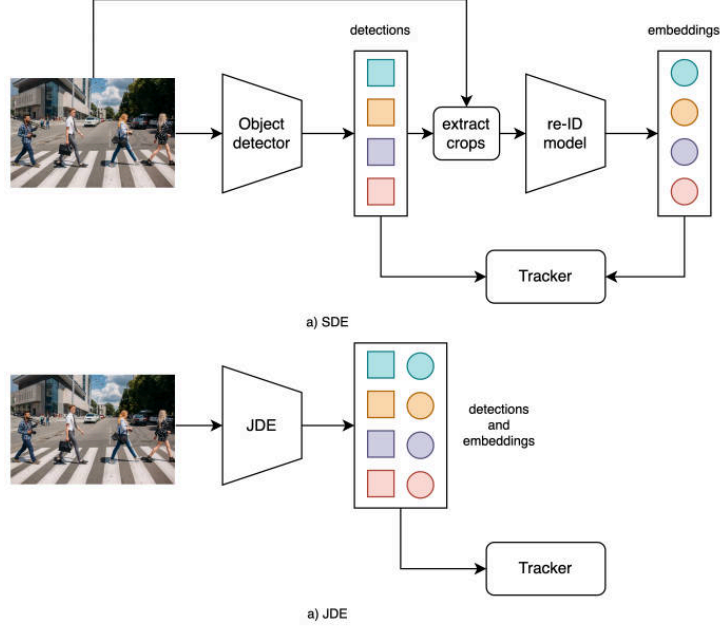


Fig. 7. Architectural comparison between joint detection-embedding (JDE) and separate detection-embedding (SDE) methods.

Instead of maintaining a buffer for a track’s historical appearance features, JDE employs an moving average over history of track’s appearance features. The track’s appearance update is as follows:

$$\mathbf{E}_{j,f+1}^T = (1 - \alpha) \cdot \mathbf{E}_{j,f+1}^D + \alpha \cdot \mathbf{E}_{j,f}^T \quad (17)$$

where α is the moving average momentum, and f is the index of the f th frame. These modifications improve tracking speed and robustness [87]. Compared to Deep SORT, this tracking method achieves approximately four times higher FPS than Deep SORT variants, making it more suitable for real-time applications. However, its tracking accuracy is lower compared to these slower models.

JDE solves the re-ID task using metric learning instead of the classification approach employed in Deep SORT [87, 89]. In the classification approach, a re-ID model is trained to assign detections to specific identities present in the dataset. In contrast, metric learning maps detections into an embedding space, where the similarity between embeddings indicates the likelihood that a track and a detection belong to the same object identity. Metric learning generally outperforms classification when used as a standalone as it is better aligned with the requirements of MOT. However, hybrid approaches that combine both metric learning and classification tend to perform very well [54, 3].

FairMOT. One major drawback of the JDE tracker is the imbalance between the re-ID and detection tasks during training, in which case the features of these tasks often fail to align. Another issue stems from architectural design, as certain priors used for object detection (e.g., anchors and feature dimensions) can introduce challenges for re-id. FairMOT [102] addresses these limitations by introducing key architectural changes, including an anchor-free CenterNet [109] detector and lower-dimensional re-ID features, thereby achieving a better balance between the two tasks. These changes led to significant improvements in tracker performance [102].

TransTrack. We classify TransTrack as a unique method that belongs to neither the tracking-by-detection nor the end-to-end paradigm but instead sits between them. Among tracking-by-detection methods, it is most similar to JDE and FairMOT.

TransTrack [77] introduces a transformer-based approach that extends the DETR [16] object detection model by adding an additional decoder specialized in track bounding box prediction. This decoder operates in parallel with DETR’s standard detection decoder, as shown in Figure 8. TransTrack forms a track query based on the track’s history and combined features from the previous and current frames. This track query is analogous to the detection query used in DETR. Based on track and detection queries, TransTrack predicts both detection and track bounding boxes. A simple IoU-based association is used to match the predicted detection and track bounding boxes. In the following frame, associated objects can be used as track queries.

Both detection and track predictions are trained using a loss similar to DETR, which combines the classification loss, bounding box L1 loss, and GIoU (Generalized intersection over union) loss [70] (instead of the IoU loss). TransTrack can be trained on static images by simulating the previous frame with random

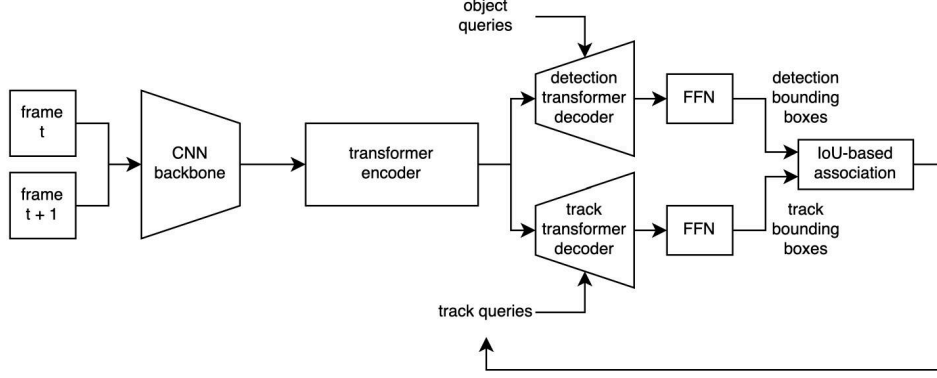


Fig. 8. TransTrack architectural design.

scaling and translations. This allows the transformer-hungry model to be trained on a combination of MOT and object detection datasets (e.g., CrowdHuman [72]).

Compared to traditional tracking-by-detection approaches, it reduces heuristic complexity by eliminating the need for advanced association methods, a re-identification model, or a KF for bounding box prediction. However, unlike fully end-to-end tracking methods, it still requires a separate association step using IoU.

4.6. Heuristic-based tracking methods

Heuristic-based methods rely solely on association heuristics to improve association accuracy. They do not introduce model-related improvements and are based on the separate detection-embedding (SDE) architecture.

ByteTrack and its derivatives. Due to its simplicity and effectiveness, *ByteTrack* has become one of the most influential tracking-by-detection methods, serving as the foundation for numerous modern approaches [74, 3, 105, 94, 61, 1, 43]. Its widespread adoption highlights its role as *the modern SORT*, providing a robust and efficient baseline that researchers continue to refine and extend. The method’s adaptability has led to various enhancements, enabling better handling of challenges such as occlusions, identity switches, and long-term tracking. As a result, ByteTrack and its derivatives are now central to state-of-the-art multi-object tracking. In the following paragraphs, we provide a detailed discussion of ByteTrack’s key innovations, its impact on the field, and the various improvements introduced by its successors.

ByteTrack. Recall that SORT’s performance is sensitive to the detection score threshold det_τ . Using a high detection threshold reduces false positives but increases false negatives, and vice versa. ByteTrack introduces a simple association method, *Byte*, that allows the use of most detections. The association process is split into two cascaded steps. In the first step, high-score detections (defined by det_τ) are associated with active and lost tracks based on motion and appearance similarity. In the second step, low-score detections are associated with the remaining (unassociated) active tracks based on motion similarity only. The main motivation is that occluded objects usually have low detection scores, as the model is less confident in these cases. Consequently, in the first step, *clearly visible* objects are prioritized for association, and subsequently, less confident detections are considered. In addition to improving tracker performance [103], the Byte association process is less sensitive to the detection score threshold compared to SORT.

ByteTrack uses IoU similarity, i.e., motion-based association, in all association cascades. A re-ID model can be used to extend the first association cascade with appearance similarity, but it is not ideal for the second association cascade. This is because low-score detections used in the second association cascade are usually occluded and have poor appearance descriptors [103]. Additionally, using a re-ID model in the second cascade would slow down inference, as it would be applied to a large number of bounding boxes. Beyond the high-score and low-score detection association steps, ByteTrack also includes a third association step for tracks in the probationary period, similar to Deep SORT. This association step takes as input new tracks and high-score detections that were not associated in the previous step. A detailed visualization of ByteTrack’s association process is shown in Figure 9.

Apart from introducing Byte, ByteTrack also employs the YOLOX object detector, which has become the default tracker for the MOTChallenge benchmark with private detections [103, 78, 21]. Further developments of ByteTrack are introduced in ByteTrackV2 [105], which unifies ByteTrack for both 2D and 3D MOT settings.

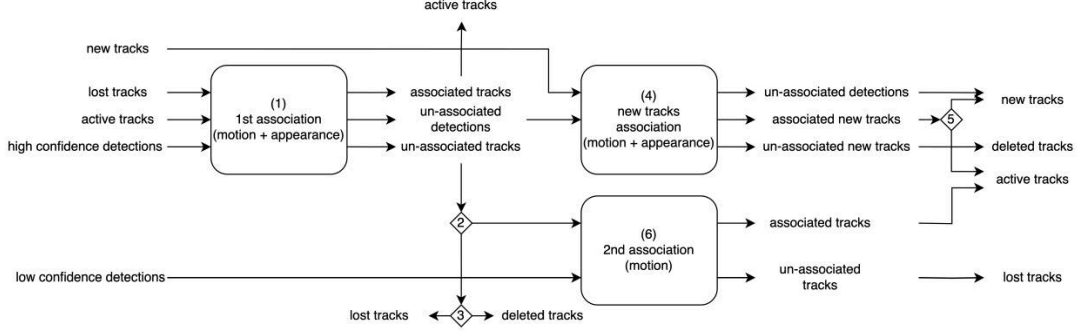


Fig. 9. ByteTrack association design. It consists of six steps, marked with numbers 1 to 6. The first association step (1), takes as input tracks with active or lost state and high-score detections. The output of this step includes associated tracks (which are further considered as active tracks), unassociated high-score detections (which are further used in step (4)), and unassociated tracks (which are further used in step (6)). Unassociated tracks are split into active and lost tracks in step (2). Lost tracks are further processed in step (3) to determine whether they should be deleted. Step (4) handles new tracks association. Unassociated detections from this step can be used as potential new tracks. Unassociated tracks are deleted as they failed during the probationary period, while associated tracks are either converted to active tracks or kept as new tracks if their probationary period has not ended (step 5). Step (6) is the second cascade of the *Byte* association approach. Active tracks that were not associated in step (1) are attempted to be associated with low-score detections. Successfully associated tracks become active, while unsuccessfully associated tracks become lost.

BoT-SORT. BoT-SORT [3] further improves ByteTrack’s performance with a few bag-of-tricks. First, BoT-SORT uses Deep SORT’s adaptive KF with time-dependent process and measurement noise approximations that scale relative to the object’s bounding box width and height. Specifically, KF predictions are more confident for smaller, distant objects that typically exhibit less displacement between frames. Second, BoT-SORT applies a camera motion compensation (CMC) method to correct KF predictions between frames in non-static camera scenes. Concretely, the global motion compensation (GMC) technique is used from OpenCV’s Video Stabilization module [11]. Additionally, it integrates FastReID’s BoT-SBS50 [54], a stronger re-identification model. This combination of techniques results in improved performance compared to the ByteTrack baseline [3]. Lastly, BoT-SORT changes the Deep SORT’s association cost defined in (15) with the new proposed one:

$$C_{\text{BoT-SORT}}(i, j) = \min\{d_M(\mathbf{D}_i, \hat{\mathbf{T}}_j), d_A(\mathbf{E}_i, \hat{\mathbf{E}}_j)\}, \quad (18)$$

where $d_A(\mathbf{E}_i, \hat{\mathbf{E}}_j)$ is defined as:

$$d_A(\mathbf{E}_i, \hat{\mathbf{E}}_j) = \begin{cases} 0.5 \cdot d_{\cos}(\mathbf{E}_i, \hat{\mathbf{E}}_j), & (d_{\cos}(\mathbf{E}_i, \hat{\mathbf{E}}_j) < t^{(A)} \ \& \ (d_M(\mathbf{D}_i, \hat{\mathbf{T}}_j) < t^{(P)}), \\ 1, & \text{otherwise} \end{cases} \quad (19)$$

where $d_{\cos}$ is the cosine distance, $\mathbf{D}_i$ is i th detection bounding box with corresponding appearance feature embedding $\mathbf{E}_i$, $\hat{\mathbf{T}}_j$ is j th track’s KF predicted bounding box, $\hat{\mathbf{E}}_j$ is the j th track’s EMA appearance feature embedding, $t^{(A)}$ is the appearance similarity threshold, and $t^{(P)}$ is the proximity threshold. Essentially, the proximity threshold is used to disable appearance-based association in cases where objects are not physically close enough. BoT-SORT also uses EMA for $\hat{\mathbf{E}}_j$ estimation [3].

ImprAsso. Although ByteTrack’s two-stage association process is highly effective, it has certain limitations. First, high-confidence detections are prioritized over low-confidence detections without considering the distances between low-confidence detections and tracks. Consequently, a detection that does not perfectly align with a track’s prediction may be preferred over a lower-confidence detection that provides a better match. Second, only active tracks, not lost tracks, are used in the second stage, which can lead to suboptimal association. ImprAsso [74] addresses these limitations by introducing a novel *combined matching* approach as an alternative to Byte. This approach integrates both high- and low-confidence detections into a single association step. Since different distance metrics can be applied to high- and low-confidence detections (e.g., appearance similarity may not be used for low-confidence detections), the distances for low-confidence detections are scaled by a factor $\beta = \frac{d_{max}^h}{d_{max}^l}$, where d_{max}^h is the maximum distance value for high-confidence detections with any track, and d_{max}^l is the maximum distance for low-confidence detections with any track.

In addition to the combined distance association method, ImprAsso proposes an occlusion-aware initialization strategy. New tracks are initialized only if they do not excessively overlap with existing tracks, as defined by an IoU threshold. The authors argue that most false-positive detections in MOT are, in fact, duplicate detections of the same person [74].

ImprAsso combines a bag of tricks from previous works. Specifically, it uses Deep SORT’s association cost function but replaces the Mahalanobis distance with IoU and employs a gate function based solely on IoU. Additionally, it uses the same re-ID model as BoT-SORT — FastReID’s BoT-SBS50—and the same CMC algorithm.

Alternative Cascaded Association Methods. Association methods generally benefit from cascades to improve tracking accuracy. Cascades are typically applied either through a two-stage process [93, 58, 86, 15] or iteratively [52, 43]. These types of heuristics are not limited to heuristic-based methods, as they are also used in motion-based trackers [61, 15, 1] and affinity learning-based methods [86, 58].

SparseTrack. SparseTrack [52] extends the BoT-SORT method by introducing cascaded associations based on objects’ pseudo-depth, i.e., the vertical position of bounding boxes, which can be used to estimate depth due to perspective projection. Objects are first sorted based on their pseudo-depth and then split into multiple cascaded levels for association. The association cost between objects is calculated using the standard IoU cost. SparseTrack’s heuristic is highly effective in heavily crowded scenes (e.g., MOTChallenge), resulting in high tracking accuracy without requiring a re-ID model.

C-BIoU. C-BIoU [93] proposes bounding box shape buffering (expansion) during association, i.e., BIoU, which increases tracking robustness to irregular motions and similar appearances. The expanded bounding box allows association based on the IoU of non-overlapping bounding boxes and compensates for motion model prediction inaccuracies. To reduce the risk of overexpansion, C-BIoU uses cascaded association with two increasing levels of expansion. This heuristic has proven to be very effective on datasets with non-linear motion [20, 21, 78]. Deep-ElIoU generalizes C-BIoU by replacing the fixed two-level cascade with an arbitrary number of levels with incremental expansion.

Confidence-Based Association Methods. Since occluded objects tend to have lower detection confidence compared to non-occluded ones, heuristics based on detection confidence can be employed during the association process. We refer to association methods that utilize such heuristics as confidence-based association methods, as they differ from motion-based and affinity-based methods. ByteTrack was one of the first methods to incorporate detection confidence during association. However, it can be further leveraged to improve association accuracy and reduce identity switches.

Hybrid-SORT. Hybrid-SORT [94] combines BoT-SORT’s baseline and motion-based heuristics from OC-SORT [15] with weak cue heuristics for more efficient occlusion handling. Hybrid-SORT introduces *tracklet confidence modeling*, where a KF is used to model bounding box confidence in addition to bounding box coordinates. This can be leveraged to associate tracks and detections based on the detection model’s confidence scores. Generally, occluded objects tend to have lower detection confidence, so incorporating confidence in the association step can help reduce the number of ID switches. These heuristics have shown great success on crowded datasets [22], as well as datasets with non-linear motion [78]. Furthermore, Hybrid-SORT proposes *height-modulated IoU (HMIoU)*, which is defined as the standard 2D IoU multiplied by height-based IoU. The motivation behind this heuristic is that objects farther from the camera in a perspective projection tend to appear smaller, even if they have a similar shape in space. The same principle can be applied to width; however, height tends to be a more stable cue [94].

BoostTrack. BoostTrack [75] introduces a tracking algorithm with a rich set of heuristics designed for crowded scenes. It employs track-detection confidence boosting (a multiplier) for similarity measures, based on the assumption that tracks and detections with high confidence scores tend to have more accurate geometric and appearance features. Track confidence is modeled as an exponential decay, meaning that lost tracks and recently initialized tracks have lower confidence scores. Track-detection confidence is computed as the product of track confidence and detection confidence, which is then used to scale the employed similarity measures based on bounding box overlap and shape differences. Additionally, BoostTrack successfully incorporates the Mahalanobis distance, originally used in Deep SORT, while mitigating its drawbacks related to the uncertainty of lost tracks. This is achieved by applying softmax to the set of Mahalanobis distances between a single track and all detections. Working with normalized distances allows the model to leverage KF’s prediction uncertainty without relying on Deep SORT’s cascaded association based on a track’s lost time. In summary, BoostTrack combines Mahalanobis distance and IoU between bounding boxes, along with scaled and L1 distances between bounding box shapes. Finally, BoostTrack introduces heuristics to increase the confidence of likely objects that may have been occluded and to decrease the confidence of unlikely objects that could be outliers. Specifically, detections with low confidence scores that have high IoU with any existing track have their confidence scores increased. Conversely, unlikely detections are ignored if their minimum Mahalanobis distance to any track exceeds a defined threshold. This combination of heuristics enables BoostTrack to achieve strong performance in extremely crowded scenes, such as those in MOT20 [22, 75].

BoostTrack++ [76] builds upon BoostTrack with an additional set of refinements, including buffered (expanded) bounding boxes [93] and further tuning of existing heuristics. These improvements lead to a slight performance gain on the MOTChallenge benchmark compared to the BoostTrack baseline [22, 76].

Object detection model exploitation. Tracktor [6] improves tracking accuracy without requiring

additional training by exploiting the Faster R-CNN detector during the association step. Specifically, Tracktor applies the detector’s regressor to the last track’s bounding box to realign it based on the features from the current frame. This approach is effective in cases where the object has moved only slightly between frames, which is often true for videos with high FPS and slow-moving objects (e.g., pedestrians). The regression precision can be further improved by using the KF’s prediction instead of the last bounding box. Additionally, Tracktor uses the detector’s classification score to decide whether a track should be terminated; a track is terminated if its classification score falls below a predefined threshold. Finally, Tracktor employs a heuristic to initialize new tracks only when no existing tracks are nearby, specifically, when the IoU between a detection (potential track) and active tracks is below a given threshold. What makes Tracktor interesting is that it requires no additional components or models. Instead, it only smartly exploiting the region-based detector — Faster R-CNN.

4.7. Motion-based tracking methods

Trackers using the linear constant velocity KF are state-of-the-art solutions on MOTChallenge [22, 76, 3, 74], which predominantly features linear motion (aligned with the KF’s assumptions). However, linear motion assumption is not valid in the general case. This is precisely why methods based on the KF underperform on datasets with dynamic and non-linear motion [78, 21, 1, 15, 91, 39]. We refer to methods that address these issues by improving the KF or replacing it with deep-learning-based models as motion-based tracking methods.

Observation centric approach. OC_SORT [15] addresses the limitations of the KF through an observation-centric⁶ approach. It introduces observation-centric re-update (ORU), which, instead of relying solely on the KF’s linear state estimation (i.e., an estimation-centric approach), handles occlusions by re-initializing the KF through interpolation of missing bounding boxes after lost track’s re-identification. In other words, if a lost track is successfully associated after N frames, then the track’s missing trajectory is approximated through interpolation. After interpolation, the KF’s state is updated for the last N frames based on the interpolated trajectory. This process corrects error accumulation in the KF state. The method is motivated by the observation that object detectors perform per-frame detection independently and do not accumulative error over time. In addition to ORU, OC_SORT proposes two more heuristics: observation-centric momentum (OCM) and observation-centric recovery (OCR). The OCM heuristic augments the association cost function by incorporating motion momentum approximated through observation history as a similarity measure. The motivation is that, over short time intervals, tracks should be associated with detections that would maintain track’s consistent motion patterns. The OCR heuristic adds an extra association step for tracks that remain un-associated, matching them with un-associated detections based on their last observed positions rather than the KF’s predicted bounding boxes. OC_SORT serves as a robust alternative to ByteTrack in modern applications, particularly in scenarios where objects exhibit non-linear motion patterns (e.g. DanceTrack or SportsMOT datasets) [15, 78, 21].

Deep OC_SORT [56] extends OC_SORT with CMC (similar to BoT_SORT), a re-ID model, and *dynamic appearance* features. Instead of using moving average to aggregate track feature embeddings over time, Deep OC_SORT introduces a modification where momentum is not constant but instead depends on the detection score (dynamic appearance). This adjustment is reasonable, as a low detection score often results from occlusion, leading to poor-quality appearance features. The Deep OC_SORT’s track’s appearance update is as follows:

$$\mathbf{E}_{j,f+1}^T = (1 - \alpha_f) \cdot \mathbf{E}_{j,f+1}^D + \alpha_f \cdot \mathbf{E}_{j,f}^T \quad (20)$$

where α_f is the moving average momentum for f th frame computed as:

$$a_{f+1} = a_f + (1 - a_f) \cdot \left(1 - \frac{c_i - \text{det}_\tau}{1 - \text{det}_\tau}\right) \quad (21)$$

where c_i is the confidence score of the i th detection, and det_τ is the detection score threshold used to filter detections. This modification does not introduce any new hyperparameters while making the appearance update more robust to low-quality appearance embeddings. In summary, Deep OC_SORT extends OC_SORT by incorporating missing components required for state-of-the-art performance on popular benchmarks.

Deep learning-based motion models. MotionTrack [61] employs a transformer-based motion model that considers an object’s history as well as its interactions with other objects. The motion model consists of two modules: an interaction module for short-range association and a refine module for long-range association. Short-range association is used for active tracks, while long-range association is used for lost

⁶In the context of MOT, KF’s observations are the output bounding boxes generated by object detection models.

tracks. The interaction module predicts an object’s motion based on its historical geometric, motion, and interaction features related to other objects present on the scene. Interaction features are extracted using a combination of the attention mechanism [82] and asymmetric convolutions [23], while their fusion is performed with a graph convolutional network [47]. For the refine module, asymmetric convolutions are also used to extract features which are then used to construct a correlation matrix between detections and lost tracks. This correlation matrix serves as input to a linear assignment process to identify tracks that are *refound*. Replacing the KF with a deep learning-based motion model that considers object interactions led to strong performance on the MOTChallenge dataset at the time.

A tracker with the same name [91] and a similar nature introduces a transformer-based motion predictor as an alternative to linear models often used in MOT. This motion predictor shows great performance when trained on a large-scale dataset — SportsMOT [21]. This tracker performs very well on the DanceTrack [78] and SportsMOT [21] datasets compared to methods that use neither a learnable motion predictor nor a strong re-ID model. However, achieving this performance requires additional datasets [21].

ETTrack [39] is another tracker based on a learnable motion predictor that combines transformers [82] and temporal convolutional network (TCN) blocks [48]. This method emphasizes motion momentum by introducing a momentum correction loss, a regularizer that tasks the model with predicting an object’s direction as well. Combined with OC_SORT’s momentum association heuristic, ETTrack demonstrates strong performance on datasets with non-linear motion [78, 21].

Another concurrent work based on a learnable motion model is MoveSORT [2]. MoveSORT replaces the commonly used KF with a deep learning-based filter that offers the same set of features, such as uncertainty estimation and noise filtering. Two families of deep learning-based filters are proposed: Bayesian and end-to-end. Bayesian filters can be used when the measurement likelihood is known or can be accurately estimated; otherwise, end-to-end variants can be used, as they simultaneously learn to predict an object’s motion and filter detection noise originating from an inaccurate detection model. Additionally, MoveSORT introduces a hybrid association cost that combines IoU similarity and L1 distance between bounding boxes during association. Essentially, L1 distance accounts for both bounding box scale and absolute position. Similar to MotionTrack [91] and ETTrack [39], MoveSORT performs well on the DanceTrack and SportsMOT [21] datasets, as it was designed for such cases.

DeepMoveSORT [1] extends MoveSORT with an improved transformer-based end-to-end filter, a re-ID model, and additional heuristics, achieving state-of-the-art performance on datasets with non-linear motion [78, 21]. However, simplicity and elegance are sacrificed compared to the MoveSORT tracker due to extensive engineering aimed at achieving strong results. This emphasizes the drawback of the standard tracking-by-detection paradigm, as many tuned components and hyperparameters are required for strong performance.

Ground-plane motion model. UCMCTrack [96] addresses challenges in video sequences with significant camera movements, in which case CMC and a re-ID model are crucial [3, 52]. The authors propose a motion-based tracker that is robust to camera movements without requiring CMC or a re-ID model. Unlike traditional methods that operate in the image plane, UCMCTrack projects objects onto the ground plane as points. Objects are projected based on their bottom-center bounding box coordinate. They use a KF with a constant velocity assumption for the ground-plane motion prediction, analogous to methods working in the image plane. They approximate measurement noise as proportional to bounding box width and height, as in [89, 3]. For process noise, they integrate camera movement. For association, they use the normalized Mahalanobis distance mapped to the ground plane, which increases distance based on uncertainty magnitude:

$$\mathbf{D} = \mathbf{e}^T \mathbf{S}^{-1} \mathbf{e} + \ln |\mathbf{S}| \quad (22)$$

where $\mathbf{e}$ is the residual between the KF’s prediction and a detection (measurement), $\mathbf{S}$ is the corresponding covariance matrix (uncertainty), and $|\mathbf{S}|$ represents the determinant of $\mathbf{S}$. The first summation component is equal to the squared Mahalanobis distance, while $\ln |\mathbf{S}|$ increases the distance based on uncertainty. This element is crucial, as it partly resolves Deep SORT’s issue related to the use of the Mahalanobis distance and increased uncertainty (which otherwise decreases distance) for lost tracks.

UCMCTrack demonstrates state-of-the-art performance on multiple datasets [49, 22, 78] with a unique approach and without using any re-ID model. However, there are some practical limitations to consider. First, while the method aims to avoid CMC, its benefits remain significant on MOT17 [96], where camera movement introduces considerable noise compared to other datasets. This suggests that in some scenarios, CMC may still be beneficial despite the method’s intended design. Second, the approach requires manually tuned camera parameters for each scene [96], which limits its adaptability to unseen environments. In other words, manual parameter tuning is necessary at test time, unlike many other methods that do not require such adjustments.

4.8. Affinity learning-based tracking methods

Affinity learning-based tracking methods improve data association by learning expressive similarity functions between detections and tracks. Unlike traditional approaches relying on handcrafted metrics, they adapt to appearance variations, occlusions, and identity switches without requiring domain-specific motion assumptions. By leveraging contrastive learning, metric learning, or graph-based models, these methods enhance tracking robustness.

Motion affinity learning. The online variant of StrongSORT [26] combines a multiple heuristics to improve the Deep SORT baseline. This includes using an improved appearance model like in [3], moving average for track appearance history aggregation like in [87], a NSA KF [25] for motion prediction, and ECC (enhanced correlation coefficient maximization) [29] for CMC. However, the offline variant—StrongSORT++—is what makes this tracking method particularly interesting. An AFLink module is trained and used to globally link tracks, addressing missed associations. This model relies only on a track’s coordinate trajectory. Trajectory features are extracted using multiple layers of 1D CNNs. The affinity between two tracks is computed by concatenating their embeddings and using an MLP to predict whether they should be associated, i.e., their connectivity. AFLink is highly efficient in resolving missed associations; however, it cannot mitigate ID switches [26]. In addition to AFLink, StrongSORT++ performs interpolation using Gaussian Process Regression (GPR) [88] instead of standard linear interpolation. Compared to linear interpolation, GPR can more accurately interpolate tracks with non-linear motion.

TWIX [58] introduces an affinity module trained to detect pairs of tracks belonging to the same object. This is achieved through a contrastive learning framework using only a track’s spatio-temporal coordinates, i.e., without appearance features. First, pairs of tracklets are encoded using a transformer with attention applied over the temporal dimension (intra-pair encoding). Second, another transformer enriches features based on other track pairs (inter-pair encoding). Finally, a linear layer predicts the pair affinity score. During inference, TWIX calculates affinity scores between historical tracks and detections. More precisely, an extended version, C-TWIX, is used, which employs cascaded matching similar to C-BIoU [93], using two different TWIX modules in the first and second association steps. What makes TWIX particularly notable is that it does not require IoU for motion-based association or any domain-specific heuristics, yet it maintains performance comparable to state-of-the-art methods across multiple datasets [58].

Appearance affinity learning. SMILETrack introduces a similarity learning module (SLM) that extracts object appearance features and predicts appearance similarity between different objects. Specifically, detection bounding boxes are used to crop object features from the frame, which are then processed using a vision transformer-based network [24]. The SLM module can be used instead of the standard re-ID model with cosine distance for appearance-based association. Apart from appearance-based association, SMILETrack’s tracking algorithm is similar to ByteTrack. Compared to the previous state-of-the-art at the time (BoT-SORT), SMILETrack improved tracker association accuracy [86].

QDTrack [30] introduces a quasi-dense similarity learning approach for multi-object tracking. Unlike traditional methods that rely on sparse ground truth matching, QDTrack densely samples numerous object regions from pairs of images to perform contrastive learning. This strategy focuses solely on appearance features, eliminating the need for motion prediction. In this framework, each detection is processed to extract appearance embeddings, which are then compared across frames using a nearest neighbor search to associate objects over time. The quasi-dense sampling enables the model to learn a more discriminative feature space, enhancing its ability to distinguish between different objects based on appearance alone. Notably, QDTrack’s similarity learning scheme is not limited to video data; it can effectively learn instance similarity from static images, allowing for competitive tracking performance without requiring video-based training or tracking supervision. Experimental results demonstrate that QDTrack achieves performance comparable to state-of-the-art methods across multiple benchmarks, including setting a new standard on the BDD100K [98] MOT benchmark, all while introducing minimal computational overhead to the detector [30].

4.9. Offline tracking methods

Offline tracking-by-detection methods require all detections to be generated beforehand, performing data association globally rather than frame-by-frame. This allows for optimal assignment strategies but prevents real-time processing. As a subset of affinity learning-based tracking, these methods construct graphs where detections are nodes and relationships are learned through similarity functions, often using Graph Neural Networks (GNNs) [108]. The GNN methods show incredible performance [17, 33] but can only be used for offline tracking. However, it’s important to note that almost every “online” method uses linear [103, 3, 52, 86, 74, 75, 76, 2, 1, 93, 94, 58, 39] or GSI (Gaussian Smoothed Interpolation) [26, 15, 56] and hence the results presented are almost always with offline post-processing.

MPNTrack. MPNTrack [12] formulates MOT as a graph-based assignment task, where link prediction is used to associate detections across frames. In this graph, detections serve as nodes, and edges

represent connections between them. MPNTrack employs a GNN with a message-passing network (MPN) architecture to model pairwise relationships between detections. The model extracts appearance features from image patches corresponding to detection bounding boxes using a CNN, which are then used as initial node features. Edge features are defined as a combination of relative geometric coordinates, appearance similarity, and time differences between detections. These node and edge features serve as inputs to the GNN framework. A message-passing network is applied to iteratively exchange information between nodes, where each step enriches nodes and edges with information from their neighbors.

The node-to-edge information update process is defined as:

$$\mathbf{h}_{i,j}^{(l)} = f_e(\mathbf{h}_i^{(l-1)} \parallel \mathbf{h}_j^{(l-1)} \parallel \mathbf{h}_{i,j}^{(l-1)}) \quad (23)$$

where l is the GNN layer corresponding to the step number, i is the index of the i th detection with the corresponding node features $\mathbf{h}_i^{(l-1)}$, (i, j) is the edge between the i th and j th detections with corresponding edge features $\mathbf{h}_{i,j}^{(l-1)}$, and f_e is a learnable function, specifically a multi-layer perceptron (MLP). The edge-to-node update process is defined as:

$$\mathbf{m}_{i,j}^{(l)} = f_v(\mathbf{h}_i^{(l-1)} \parallel \mathbf{h}_{i,j}^{(l)}) \quad (24)$$

where f_v is another learnable MLP function. The final node embeddings are obtained through an aggregation step:

$$\mathbf{h}_i^{(l)} = \Phi(\{\mathbf{m}_{i,j}^{(l)}\}_{j \in \mathbf{N}_i}) \quad (25)$$

where $\mathbf{N}_i$ represents the neighbors of the i th detection (node), and Φ is an order-invariant aggregation function, such as summation. Equations (23), (24), and (25) define the message-passing framework. To account for temporal dependencies, MPNTrack aggregates information separately from past and future detections in (24) using different networks before combining them in a *time-aware message-passing* mechanism.

Since the graph scales exponentially with video length, an edge between two nodes is retained only if both are among the top 50 mutual nearest neighbors based on appearance similarity. MPNTrack’s joint modeling of visual and geometric cues improves robustness to occlusions and reduces identity switches. The method processes video clips of 15 frames with a stride of 1 and merges predictions post-processing [12].

SUSHI. SUSHI [17] extends MPNTrack by replacing the monolithic graph structure with a hierarchy of graphs. In the first step, pairs of frames are associated based on their relative detection features. A SUSHI block is used to associate detections, resulting in tracks with a maximum length of 2. In the next step, a graph of these tracks is formed, with the maximum length of each track extended to 4. This process is repeated 9 times, producing tracks with a maximum length of 512 frames. The SUSHI block is essentially similar to MPNTrack, except it additionally incorporates motion features as inputs alongside geometric and appearance features. By sharing weights across the SUSHI blocks, the number of training samples is effectively increased, leading to empirically improved performance and better convergence. Similar to MPNTrack, only the 10 nearest neighbors are considered for each node based on geometric, motion, and appearance similarity. Videos are processed offline as clips of length 512 with a stride of 256. These modifications to the original MPNTrack method result in a significant performance boost across multiple datasets [17].

CoNo-Link. When formulating MOT as a graph-based assignment problem, the size of the graph grows exponentially with video length. Learning associations through these graphs is infeasible in terms of both computation and memory [12, 17, 33]. To mitigate this, edge pruning based on appearance features is necessary. In the case of MPNTrack, retaining the top 50 neighbors per node is required to maintain 99.9% envelopment. The risk of mistakenly pruning crucial edges still exists even if many neighbors are retained. Instead, CoNo-Link introduces *NodeNet*, a transformer-based neural network that models the association likelihood between two detections (or a track and a detection). Using these likelihoods, a pruned graph is formed, preserving only essential edges (e.g. 5 to 10). This pruned graph is constructed over a video clip using a tracking-by-detection approach, where tracks from previous frames (with their detections) are associated with detections in the current frame. The similarity between a track and a detection is computed as the sum of all association scores between the track’s detections and the detection in the current frame. In addition to the NodeNet association score, IoU similarity is also incorporated. The resulting graph is used as input to a GNN, which integrates tracks obtained from NodeNet into full video tracks. Compared to other approaches, CoNo-Link incorporates detection-to-detection, detection-to-track, and track-to-track edges in the constructed graph. The final tracking solution is derived by aggregating track-to-track links. CoNo-Link is a more complex solution compared to MPNTrack and SUSHI, but it outperforms them on multiple datasets [33].

4.10. Tracking-by-detection discussion

The main benefit of the tracking-by-detection paradigm is its modularity. Each component (e.g., object detector, motion model, association, etc.) can be replaced independently, making it highly adaptable to specific task requirements. This flexibility has resulted in a wide range of tracking methods. For benchmark datasets, a slower and larger object detector, such as YOLOX-X, with many parameters and high resolution, is typically used to achieve the highest tracking accuracy. On the other hand, for tasks requiring real-time performance, a smaller object detection model (e.g., YOLOX-S) can be employed [34]. If objects’ motion is noisy due to unpredictable camera movement, a camera motion compensation (CMC) module should be employed [3, 89, 26]. In domains where objects exhibit dynamic and nonlinear motion, deep learning-based motion models combined with a strong re-ID model are crucial [56, 1]. Last but not least, association accuracy can be improved with a set of heuristics based on domain knowledge.

The main drawback of current tracking-by-detection methods are their dataset-specific heuristics. For instance, the standard KF is widely used as the default motion model [8, 89, 103, 3, 105, 15, 75, 94, 52, 6]. However, it struggles with modeling non-linear motion and requires domain knowledge for parameter tuning. Additionally, many tracking methods rely heavily on domain-specific association heuristics rather than leveraging general machine learning models capable of learning from the data.

We argue that this limitation stems from the historically limited variety of datasets available for tracker evaluation. MOTChallenge, the most popular MOT benchmark, is specialized for pedestrian tracking [22]. However, with the introduction of additional large-scale datasets, research is shifting towards more general solutions — end-to-end tracking [99, 104, 97, 14, 31, 32].

We argue that affinity learning, including offline GNN-based methods, represents the most natural progression for the tracking-by-detection paradigm from the perspective of deep learning and machine learning. These methods are capable of learning directly from data while preserving modularity and flexibility through a separate object detection model.

5. End-to-end tracking paradigm

We define a tracking method as end-to-end if object detection and association are trained jointly. Such methods require no post-processing after association, and the only tracker logic and hyperparameters pertain to track handling (track initialization, track deletion, etc.). Currently, most well known end-to-end multi-object tracking methods are extensions of the DETR [16] end-to-end object detection model [99, 104, 97, 14, 31, 32, 57], which was detailed in Section 4. We first introduce MOTR (end-to-end Multi-Object Tracking with Transformer) [99] along with its modifications to DETR. This provides a foundation for understanding other end-to-end methods, which are similar to MOTR. MOTR and related approaches [104, 97, 14, 31] are commonly referred to as tracking-by-query MOT trackers.

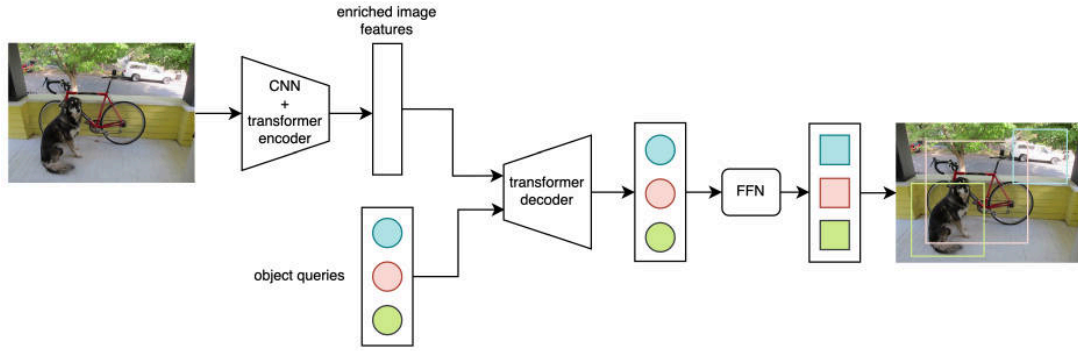
5.1. End-to-end tracking methods

MOTR. We introduce MOTR as an extension of DETR for multi-object tracking. As illustrated in Figure 10, the DETR pipeline proceeds as follows: it takes an image as input and extracts features using a CNN backbone. These features are further enriched using a transformer encoder. DETR’s decoder predicts object embeddings via an attention mechanism, using the image features (key and value) and learnable detect queries as input. The predicted embeddings are then used to derive objects’ bounding boxes and classes.

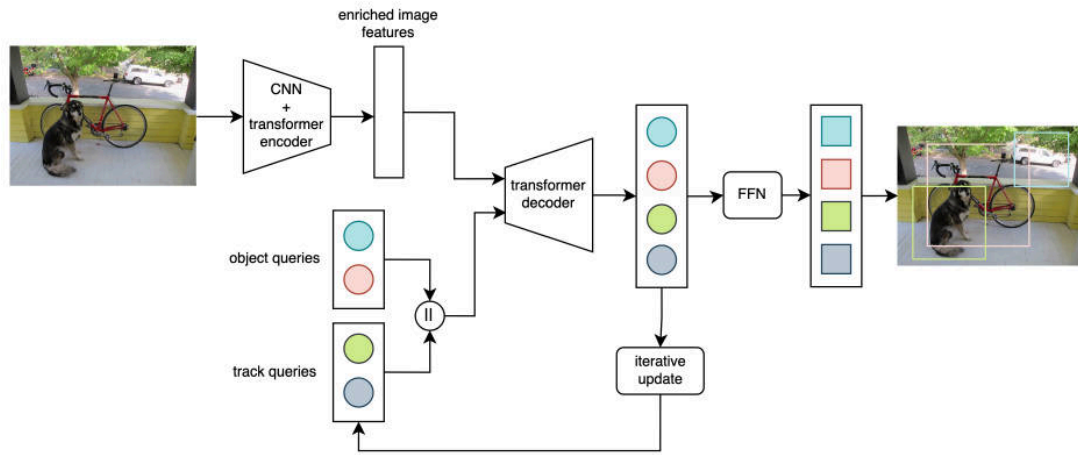
As shown in Figure 10, MOTR introduces two key modifications to the DETR architecture: *track queries* and their *iterative updates*. In addition to detect queries, which identify new objects, MOTR extends the query set with track queries that encode an object’s historical context. These track queries are responsible for detecting tracked objects in the current frame. At a high level, MOTR can be seen as an extension of DETR that incorporates temporal context from previous frames. As an object’s state evolves over time, its corresponding track query is iteratively updated, maintaining a continuously refined hidden state.

Successful detection of a tracked object requires updating its query based on the current frame’s features. This is accomplished through a *Temporal Aggregation Network* (TAN), which enhances temporal relationship modeling. Specifically, track queries from the current and previous frames are aggregated using an attention mechanism, followed by an MLP and normalization layers. In theory, this allows the model to aggregate a track’s context over time in an autoregressive manner [99].

Training. MOTR extends DETR’s bipartite matching training approach. For newborn objects, bipartite matching is performed in the same way as in DETR, where detect queries are assigned to the ground truths of newly detected objects. On the other hand, track query assignments are consistent with ground truth IDs from the previous frames.



a) DETR end-to-end object detection model



b) MOTR end-to-end object tracking model

Fig. 10. Comparison of DETR and MOTR architectures. (a) DETR uses object queries with a Transformer decoder for end-to-end object detection. (b) MOTR extends DETR by adding track queries, enabling iterative updates for multi-object tracking.

Training is conducted on video clips of varying lengths. Predictions from the full clip are used to compute frame losses, which are then averaged before backpropagation. The loss consists of three components, similar to DETR [16]: classification, L1, and GIoU loss [70]. The final loss is calculated as a weighted sum of these components.

Inference. During inference, MOTR processes videos in a fully end-to-end manner, without requiring any additional logic after each frame’s inference [99]. In the initial frame, only detect queries are used. High-confidence detections are used to initialize tracks and their corresponding track queries. In subsequent frames, the model uses track queries to predict the new positions of tracked objects and detect queries to identify new objects. The model is designed to handle duplicate detections robustly (e.g., the same object should not be detected by both detect and track queries) [16, 99]. Additionally, if a track’s confidence drops below a certain threshold, it is removed from the track set [99]. The number of detection queries is fixed and set to a value larger than the maximum number of objects expected in any given frame of the dataset, while the number of track queries varies dynamically over time [16, 99].

TrackFormer. A method similar to MOTR is TrackFormer [57], which relies on NMS post-processing, preventing it from being a fully compact end-to-end solution. Additionally, TrackFormer does not use an advanced iterative update network like TAN and instead relies on embedding predictions from the previous frame.

TransTrack [77], mentioned in Section 4, shares similarities with TrackFormer and MOTR. The transition from TransTrack to TrackFormer involves merging the decoders, resulting in integrated association, as seen in TrackFormer. The transition from TrackFormer to MOTR entails removing the post-processing step after model inference.

MOTRv2. Compared to tracking-by-detection methods, which rely on standalone object detections, the main limitation of MOTR is its poor detection performance. This issue stems from having too few supervision labels for detection queries, as only newborn objects are considered [99, 104, 97]. MOTRv2 [104] addresses this limitation by introducing YOLOX, which generates a set of bounding box proposals that can be used as anchors for detection queries within the MOTR framework. In this approach, it is sufficient to predict anchor-relative offsets instead of absolute bounding box coordinates. This modification significantly improves MOTR’s performance across multiple datasets. However, it represents a step backward from the end-to-end approach, as it requires an independently trained object detection model during inference. MOTRv2 does not satisfy our definition of an end-to-end tracker and cannot be considered one. However, it is included in this section as it is based on MOTR.

MOTRv3. MOTRv3 [99] overcomes MOTR’s limitations without including an additional object detection network during inference, unlike MOTRv2. MOTRv3 is a direct successor to the original MOTR and is independent of the modifications introduced in MOTRv2. The modifications compared to MOTR are as follows. First, they modify the bipartite matching between (detection and track) queries and ground truths during training. They eliminate the use of track query assignments based on ground-truth IDs from previous frames, relying instead on matching costs such as class labels and bounding box distances. This adjustment allows the training process to automatically balance assignments between track and detection queries. As a result, detection query assignments are more frequent at the start of training and decrease as training progresses. This leads to a significant improvement in detection performance compared to the original MOTR [97]. Notably, the supervision for the last MOTR decoder layer remains unchanged from the original paper. Secondly, MOTRv3 proposes leveraging YOLOX outputs for pseudo-label distillation. Unlike MOTRv2, the additional object detection model is only used during training and not during inference. Lastly, inspired by Group DETR [18], MOTRv3 employs track group queries instead of track queries to further enhance supervision. With these modifications, MOTRv3 achieves tracking performance on par with MOTRv2 while maintaining an elegant end-to-end training and inference framework.

MeMOT. Compared to MOTR, which iteratively updates track queries, MeMOT (Multi-Object Tracking with Memory) [14] uses a spatio-temporal memory buffer that stores the history of track states, i.e., track embeddings. Memory buffer is aggregated using a transformer-based memory aggregator which encodes separately both short-term and long-term temporal dependencies. These dependencies are then aggregated into track queries and memory updates. Detect queries are generated through a transformer decoder proposal network. Finally, detect and track queries, with image features, are used as inputs to the (memory) decoder that predicts track’s bounding boxes, objectness, i.e. object visibility, and uniqueness, i.e. if the object is unique. Objectness and uniqueness scores are used in track handling logic to discard background and duplicate objects.

MeMOTR. MeMOTR [31] is an end-to-end tracker most similar to the MeMOT tracking method. The primary difference is that, instead of a memory buffer, MeMOTR uses a long-term memory embedding — a running average of the decoder outputs. Unlike MeMOT, where the aggregated memory buffer is used for queries, this long-term memory embedding is used as the key in the transformer decoder, replacing image features.

MOTIP. MOTIP (Multi-Object Tracking as ID Prediction) [32] differs from tracking-by-query

methods, i.e., MOTR and MeMOT variants, as it addresses the MOT problem from a different perspective. Tracking-by-query methods can be viewed as extended DETR object detectors with context from previous frames. In contrast, given a set of tracks’ state histories (DETR output history), MOTIP assigns the ID of the most similar track to each detected object. The MOTIP decoder uses tracks’ state histories as keys and values, and DETR outputs, i.e., detected object embeddings, as queries. The decoder classifies the ID of each detection based on the list of all track IDs. MOTIP employs a learnable ID dictionary with keys assigned to each track. Predicting learnable keys, instead of fixed one-hot label representations, enhances the model’s usability for long videos, where the total number of IDs can be very large. Assignments based on MOTIP outputs are performed using either greedy assignment or the Hungarian algorithm. We can observe that MOTIP shares its similarity with affinity learning methods. MOTIP’s performance is on par with the best tracking-by-query methods [32].

5.2. End-to-end tracking discussion

The mentioned end-to-end tracking methods, primarily related to the tracking-by-query paradigm, offer fully differentiable models that can be jointly optimized for detection, association, and motion modeling in a single training phase. Since these solutions learn to track directly from data, they are not domain-specific and provide a generalizable approach. However, they come with significant computational demands. First, they require large numbers of GPUs with high VRAM [99, 104, 97, 14, 31, 32] for training, making them inaccessible to most researchers and industry practitioners. Second, they are slow during inference, which prevents their use in real-time applications. Currently, tracking-by-detection methods are generally more suitable for real-time object tracking. Lastly, they generally perform poorly compared to tracking-by-detection methods on heavily crowded scenes [99, 14, 104, 97, 31, 32].

6. Methods comparison

We compare state-of-the-art methods on the previously mentioned benchmark datasets, dividing them into two groups: (1) highly crowded scenes with linear object motion (MOT17 and MOT20) and (2) moderately crowded scenes with dynamic motion and similar object appearances (DanceTrack, SportsMOT). We categorize MOT methods into three paradigms: *end-to-end*, *tracking-by-detection*, and *offline* (a subset of tracking-by-detection). In this context, *offline* refers to methods that perform association after all detections have been obtained.

MOTChallenge benchmark. Comparison of state-of-the-art methods on MOT17 and MOT20 are presented in Table 3 and Table 4, respectively. Additionally, Figure 11 provides a visual comparison of these methods on both MOT17 and MOT20 datasets. Over the past three years (2022–2024), tracking performance in terms of HOTA has improved by 4.0%, i.e. from 63.1% (ByteTrack) to 67.1% (CoNo-Link) on the MOT17 dataset and 5.1%, i.e. from 61.3% (ByteTrack) to 66.4% (BoostTrack++) on the MOT20 dataset. When considering only online methods, the most improvement has been achieved with the BoostTrack++ tracker, with 3.4% and 5.1% improvements in terms of HOTA on MOT17 and MOT20, respectively.

From Figure 11, we observe that BoostTrack++, CoNo-Link, and ImprAsso stand out as they are Pareto-optimal in terms of either HOTA or MOTA. Both BoostTrack++ and ImprAsso are SORT-like methods that achieve strong performance through combination motion-, appearance-, and detection confidence-based associations. These association components are employed through a rich set of heuristics, which allow them to efficiently handle crowded scenes in the MOTChallenge datasets. However, these trackers lack deep learning-based improvements relative to earlier approaches (e.g., ByteTrack, BoT-SORT), as they rely on the same object detection and Re-ID models. The CoNo-Link offline tracking method stands out due to its strong performance, achieved by association techniques combining transformers [82] and graph neural networks (GNNs) [108]. Unlike BoostTrack++ and ImprAsso, whose performance depends on manually tuned hyperparameters, CoNo-Link’s performance scales with dataset size and quality. A noteworthy offline alternative to CoNo-Link is SUSHI, a simpler yet effective approach.

Based on the results in Table 3 and Table 4, we can conclude that end-to-end MOT methods perform worse compared to tracking-by-detection methods on the MOTChallenge benchmark, which includes highly crowded scenes. Results for end-to-end methods are typically reported only on MOT17 and are often missing for MOT20 [99, 104, 97, 14, 31, 32], which features even more crowded scenarios. However, based on the presented results in Table 4 and the similar nature of all end-to-end methods, we can assume that these methods generally perform poorly on MOT20 compared to tracking-by-detection approaches employing YOLOX object detector. The authors of MOTRv2 highlight that one of the main challenges of MOT17 for training transformer-based models is the small dataset size in terms of video duration [104] — 463 seconds for MOT17 and 535 seconds for MOT20. Additionally, in MOTIP, it is noted that this issue may be related to the limitations of transformer-based detectors when detecting small and densely packed

Method	Year	HOTA	AssA	DetA	MOTA	IDF1	IDSW
<i>offline</i>							
SUSHI* [17]	2022	<i>66.5</i>	<i>67.8</i>	<i>65.5</i>	81.1	<i>83.1</i>	1149
CoNo-Link* [33]	2024	67.1	<i>67.8</i>	66.7	82.7	83.7	<u>1092</u>
<i>tracking-by-detection</i>							
ByteTrack [103]	2022	63.1	62.0	64.5	80.3	77.3	2196
BoT-SORT [3]	2022	65.0	65.5	64.9	80.5	80.2	1212
OC_SORT [15]	2023	63.2	63.4	63.2	78.0	77.5	1950
ByteTrackV2 [105]	2023	63.6	62.7	65.0	80.6	78.9	1239
StrongSORT++ [26]	2023	64.4	64.4	64.6	79.6	79.5	1194
Deep_OC_SORT [56]	2023	64.9	65.9	64.1	79.4	80.6	<i>1023</i>
MotionTrack [61]	2023	65.1	65.1	65.4	81.1	80.1	1140
SparseTrack [52]	2023	65.1	65.1	65.3	81.0	80.1	1170
ImprAsso [74]	2023	66.4	66.6	66.4	<i>82.2</i>	82.1	924
C-BIoU [93]	2023	64.1	63.7	64.8	81.1	79.7	1194
ETTrack [39]	2024	61.9	60.5		79.0	75.9	2188
MoveSORT [2]	2024	62.2	60.4	64.3	79.5	75.1	2688
DeepMoveSORT [1]	2024	63.2	62.8	63.8	78.7	77.3	1524
Hybrid-SORT [94]	2024	64.0			79.9	78.7	
SMILETrack [86]	2024	65.3			81.1	80.5	1246
UCMCTrack [△] [96]	2024	65.7	66.4	65.3	80.6	81.0	1689
BoostTrack+ [75]	2024	66.4	<i>67.7</i>	65.4	80.6	81.8	1086
BoostTrack++ [76]	2024	<u>66.6</u>	68.0	<i>65.5</i>	80.7	<i>82.2</i>	1062
<i>end-to-end</i>							
MeMOT [14]	2022	56.9	55.2		72.5	69.0	2724
MOTR [99]	2022	57.8	55.7	60.3	68.6	73.4	2439
MeMOTR [31]	2023	58.8	58.4	59.6	72.8	71.5	<u>1902</u>
MOTRv3 [97]	2023	60.2	58.7	62.1	75.9	72.4	2403
MOTRv2 [104]	2023	<u>62.0</u>	<u>60.6</u>	<u>63.8</u>	<u>78.6</u>	<u>75.0</u>	
MOTIP [32]	2024	<u>59.2</u>	56.9	62.0	<u>75.5</u>	71.2	

Table 3: Comparison of tracking methods on the MOT17 benchmark. All tracking-by-detection methods use the same private detector — ByteTrack’s YOLOX-X object detection model [103]. The best results for each metric are highlighted in bold, and the second-best results for each metric are highlighted in italics. The best results for each paradigm separately are underlined. Methods marked with Δ employ test set information. The results are sourced from the MOTChallenge [22] public leaderboard, except for Hybrid-SORT, ETTrack, SMILETrack, MOTRv2, MOTRv3, MeMOT, and MoTIP, in which case the results are taken from the paper.

objects [16, 110, 32]. For MOTRv2, the best-performing end-to-end tracker on MOT17, the use of the YOLO object detector for detection proposals partly mitigates DETR’s limitations.

Finally, we discuss the effectiveness of the most important association components of tracking-by-detection methods: motion model, re-ID model, CMC (Camera Motion Compensation), and association heuristics. Online state-of-the-art trackers on MOT17 and MOT20 use the standard KF, which includes a linear motion model with a constant velocity assumption [75, 76, 74, 96, 94, 93, 52, 56, 15, 103, 3]. As pedestrian motion in this case is linear and predictable, a learnable motion model [91, 39, 2, 1] is expected to be unhelpful for this domain. Replacing the KF with a deep learning-based motion model in this context leads to suboptimal tracker performance, as evidenced by the performance of MoveSORT, ETTrack, and DeepMoveSORT, as well as the ablation studies in [1]. On the other hand, every performant method uses a re-ID model. However, the re-ID model does not have a significant effect in this case, as can be seen when comparing the metrics of BoT-SORT and BoT-SORT-REID in [3]. Using a re-ID model results in 0.4% and 0.7% performance improvement in terms of HOTA on MOT17 and MOT20, respectively. Similar conclusions can be drawn from other ablation studies [103, 56, 52]. A more crucial component, particularly on MOT17 with frequent and unpredictable camera movements, is the CMC [3, 56, 1, 61, 52, 75]. Integrating CMC (e.g., GMC from the OpenCV library [11]) into a tracker on MOT17 can lead to a 1.0% to 1.5% improvement in terms of HOTA [3, 56, 1]. Since MOT20 primarily consists of static cameras, CMC is not as relevant [56]. Lastly, the addition of a rich set of heuristics significantly enhances tracking performance on both datasets compared to the simple ByteTrack baseline. Some of these heuristics are based on pseudo-depth [52, 1], detector confidence [103, 94, 75, 76, 1], object size [94, 75, 2, 1], bounding box expansions [93, 1], motion momentum [94, 15, 56], and more.

DanceTrack Benchmark. A comparison of various tracking methods on the DanceTrack dataset is presented in Table 5. Compared to MOTChallenge, we observe a significant shift in the nature of the results. First, considering improvements relative to the strong ByteTrack baseline, the total improvement is 19.8%, i.e., from 53.6% to 73.4%, in terms of HOTA (MOTRv2), 21.7%, i.e., from 55.3% to 76.0%, in terms of IDF1 (MOTRv2), and 0.6%, i.e., from 92.3% to 92.9%, in terms of MOTA (MOTRv3). Since

Method	Year	HOTA	AssA	DetA	MOTA	IDF1	IDSW
<i>offline</i>							
SUSHI [17]	2022	64.3	67.5	61.5	74.3	79.8	706
CoNo-Link [33]	2024	<u>65.9</u>	<u>68.0</u>	<u>64.0</u>	<u>77.5</u>	<u>81.8</u>	956
<i>tracking-by-detection</i>							
ByteTrack [103]	2022	61.3	59.6	63.4	77.8	75.2	1223
BoT-SORT [3]	2022	63.3	62.9	64.0	77.8	77.5	1313
ByteTrackV2 [105]	2023	61.4	60.1	62.9	77.3	75.6	1082
OC_SORT [15]	2023	62.4	62.5	62.4	75.7	76.3	942
StrongSORT++ [26]	2023	62.6	64.0	61.3	73.8	77.0	1003
MotionTrack [61]	2023	62.8	61.8	64.0	<i>78.0</i>	76.5	1165
SparseTrack [52]	2023	63.5	63.1	64.1	78.1	77.6	1120
Deep OC_SORT [56]	2023	63.9	65.7	62.4	75.6	79.2	779
ImprAsso [74]	2023	<i>64.6</i>	64.6	64.8	78.6	78.8	992
MoveSORT [2]	2024	60.5	60.0	61.3	74.3	74.0	943
DeepMoveSORT [1]	2024	60.6	60.8	60.5	73.6	74.1	1180
UCMCTrack [△] [96]	2024	62.8	63.5	62.4	75.6	77.4	1355
C-TWIX [58]	2024	63.1	62.5	<i>64.1</i>	78.1	76.3	5820
SMILETrack [86]	2024	63.4			<i>78.2</i>	77.5	1208
Hybrid-SORT [94]	2024	63.9			76.7	78.4	
BoostTrack+ [75]	2024	<i>66.2</i>	<i>68.6</i>	<i>64.1</i>	77.2	81.5	827
BoostTrack++ [76]	2024	66.4	68.7	64.3	77.0	82.0	<i>762</i>
<i>end-to-end</i>							
MeMOT [14]	2022	54.1	55.0		63.7	66.1	<u>1938</u>
MOTRv2 [104]	2023	<u>61.0</u>	<u>59.3</u>	<u>63.0</u>	<u>73.1</u>	<u>76.2</u>	

Table 4: Comparison of tracking methods on the MOT20 benchmark using a private detector (ByteTrack’s YOLOX-X object detection model [103]). The best results for each metric are highlighted in bold, and the second-best results for each metric are highlighted in italics. The best results for each paradigm separately are underlined. Methods marked with Δ employ test set information. The results are sourced from the MOTChallenge [22] public leaderboard, except for Hybrid-SORT, SMILETrack, MeMOT, and MOTRv2, in which case the results are taken from the paper.

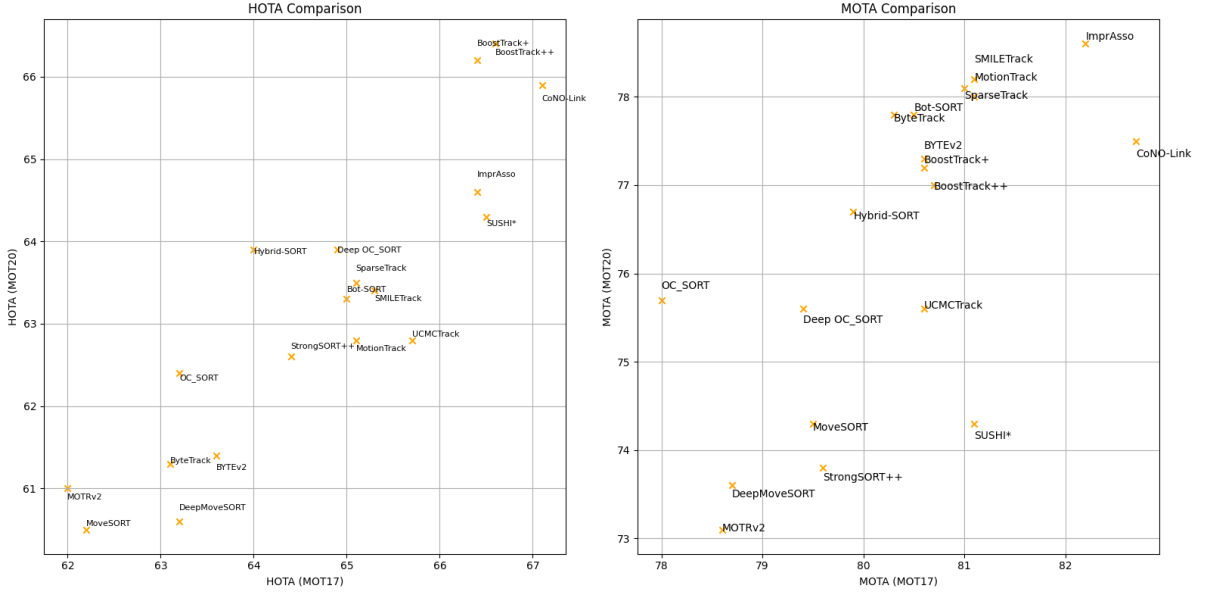


Fig. 11. Comparison of HOTA and MOTA scores between MOT17 and MOT20 benchmarks for various tracking methods. The left plot shows the relationship between HOTA scores, while the right plot highlights MOTA scores. Methods are annotated with their names, and slight vertical offsets help distinguish overlapping points.

ByteTrack’s performance in terms of MOTA is as high as 92.3%, the relative improvement is expected to be lower compared to HOTA, which is only 53.6%. As shown in the table, these performance gains stem from the introduction of new end-to-end methods. Besides the original MOTR trackers, every end-to-end tracker significantly outperforms all tracking-by-detection methods. Specifically, MeMOTR, which has the lowest performance among end-to-end trackers, still outperforms CoNO-Link, the best-performing tracking-by-detection (online and offline) tracker, by 4.7% in terms of HOTA. Concretely, MOTRv2 achieves the best tracking performance, with 73.4% HOTA, 92.1% MOTA, and 76.0% IDF1. As MOTRv2 is not a *true end-to-end* method, we also highlight the second-best method, MOTIP, with 71.4% HOTA, 91.6% MOTA, and 76.3% IDF1. It is important to note that all tracking-by-detection and offline methods use the same public YOLOX detector [78], while the end-to-end methods train private detectors. This implies that some improvements come directly from the quality of the detections, which can be partially observed through the DetA metric. For an ideal and isolated association performance comparison, it would be necessary to use the DETR detector’s outputs instead of the YOLOX detector for tracking-by-detection method evaluation. Unfortunately, this information is not available in the existing literature.

From Table 5, we also observe that SORT-like methods with deep learning-based motion models (e.g., MoveSORT, ETTrack, MotionTrack) prove highly effective on the DanceTrack dataset. Even without appearance features or a rich set of heuristics, these methods improve the DanceTrack benchmark baselines. For instance, compared to ByteTrack, MotionTrack improves performance by 4.6% (i.e., from 53.6% to 58.2%) in terms of HOTA. Furthermore, combining improved motion and re-ID models (e.g., Deep OC_SORT, DeepMoveSORT) leads to even stronger performance. Specifically, compared to ByteTrack, DeepMoveSORT achieves a 9.4% improvement (i.e., from 53.6% to 63.0%) in terms of HOTA. The best-performing online tracking-by-detection model is UCMCTrack⁷, which employs video consistent CMC [96] and performs association in a mapped ground plane. The best offline variant is CoNo-Link, a GNN based method with 63.8% HOTA.

As with MOTChallenge, we discuss the effectiveness of the most important association components of tracking-by-detection methods: motion model, re-ID model, CMC, and association heuristics. In the previous paragraph, we analyzed the effectiveness of learnable deep learning-based motion models [39, 2, 1, 91]. Even for baselines such as SORT and ByteTrack, replacing the KF with stronger motion models improves HOTA by up to 4.8% [1] on the DanceTrack validation set. This is expected, as the dancers in this dataset exhibit dynamic, non-linear motion, which the KF fails to model effectively. We also observe the importance of the re-ID model, which strongly influences performance, as demonstrated in the ablations in [56, 1]. In [1] ablation study, it can be seen that the performance of ByteTrack on the DanceTrack validation set improves by up to 5.2% (i.e., from 53.8% to 59.0%) in terms of HOTA. At first glance, this is unexpected since the dancers typically wear similar uniforms. While this makes appearance-based association challenging, it is far from obsolete. The use of CMC is noted in [56], but it is not as critical as in the MOT17 benchmark. Strong motion or re-ID models suffice for cases with non-static cameras. Lastly, heuristics are also beneficial on DanceTrack, and their importance is on par with re-ID and motion models [15, 56, 52, 2, 1, 39, 94].

SportsMOT Benchmark. A comparison of various tracking methods on the SportsMOT dataset is presented in Table 6. As SportsMOT is a less-known benchmark compared to DanceTrack and MOTChallenge, evaluations for some important methods are missing. Compared to the ByteTrack baseline, we observe a total improvement of 14.6% (DeepMoveSORT), i.e., from 64.1% to 78.7%, in terms of HOTA; 2.3% (ETTrack), i.e., from 94.5% to 96.8%, in terms of MOTA; and 10.3% (DeepMoveSORT), i.e., from 71.4% to 81.7%, in terms of IDF1. In this case, tracking-by-detection methods outperform end-to-end methods by up to 6.8% in terms of HOTA. However, this comparison is not entirely valid, as the YOLOX detector used by tracking-by-detection methods is trained on both the train and validation sets for evaluation on the test set [21, 43, 39, 2, 1, 91], whereas end-to-end methods are trained only on the training dataset [31, 32].

We discuss the effectiveness of the most important association components of tracking-by-detection methods: motion model, re-ID model, CMC, and association heuristics. In this case, there is a notable difference in performance between ByteTrack, which does not use a re-ID model in this evaluation, and BoT-SORT, which does. The BoT-SORT tracker outperforms ByteTrack by 4.6%, i.e., from 64.1% to 68.7%. The importance of appearance features is further confirmed through ablation studies in [43, 1]. Since the motion in SportsMOT is even more dynamic compared to DanceTrack, due to both player and camera movements, tracking methods with deep learning-based motion models prove even more effective. This is evident from the improved performance of MoveSORT, ETTrack, and MotionTrack compared to ByteTrack. Additionally, OC_SORT, which partially resolves some issues of the KF and thus enhances its capabilities, demonstrates very strong performance despite relying solely on the object detector, the KF,

⁷UCMCTrack requires manually configured camera parameters for each scene, even on the test set.

Method	Year	HOTA	AssA	DetA	MOTA	IDF1
<i>offline</i>						
SUSHI* [17]	2022	63.3	50.1	80.1	87.7	63.3
CoNo-Link* [33]	2024	<u>63.8</u>	<u>50.7</u>	<u>80.2</u>	<u>89.7</u>	<u>64.1</u>
<i>tracking-by-detection</i>						
ByteTrack [103]	2022	53.6	36.1	79.9	<i>92.3</i>	55.3
BoT-SORT [103]	2022	54.2	36.8	80.0	92.1	57.0
OC.SORT [15]	2023	55.1	38.0	80.4	89.4	54.9
SparseTrack [52]	2023	55.5	39.1	78.9	91.3	58.3
C-BIoU [93]	2023	60.6	45.4	81.3	91.6	61.6
Deep OC.SORT [56]	2023	61.3	45.8	81.6	91.8	61.5
MoveSORT [2]	2024	56.1	38.7	81.6	91.8	56.0
ETTrack [39]	2024	56.4	39.1	81.7	92.2	57.5
MotionTrack [91]	2024	58.2	41.7	81.4	91.3	58.6
C-TWIX [58]	2024	62.1	47.2	81.8	91.4	63.6
DeepMoveSORT [1]	2024	63.0	48.6	<u>82.0</u>	<u>92.6</u>	<u>65.0</u>
UCMCTrack [△] [96]	2024	<u>63.6</u>	<u>51.3</u>	78.9	88.9	<u>65.0</u>
<i>end-to-end</i>						
MOTR [99]	2022	54.2	40.2	73.5	79.7	51.5
MeMOTR [31]	2023	68.5	58.4	80.5	89.9	71.2
MOTRv3 [97]	2023	70.4	59.3	83.8	92.9	72.3
MOTRv2 [104]	2023	73.4	64.4	<i>83.7</i>	92.1	<i>76.0</i>
MOTIP [32]	2024	<i>71.4</i>	<i>62.8</i>	81.3	91.6	76.3

Table 5: Comparison of tracking methods on the DanceTrack benchmark using the public DanceTrack [78] detector. The best results for each metric are highlighted in bold, while the second-best results are highlighted in italics. The best results for each paradigm separately are underlined. Methods marked with * are restricted to offline mode, while those marked with Δ employ test set information. The results SORT, ByteTrack and BoT-SORT were produced by us. All other results are sourced from their respective published papers.

and observation-centric heuristics [15]. For this domain, CMC methods are not as critical since the camera motion is predictable. Lastly, most association-related heuristics are less effective [1], primarily due to the sparsity of objects in the scene, where occlusions are less frequent compared to other datasets [21]. Notably, bounding box expansion before association proves to be highly effective, as tracks’ bounding boxes between consecutive frames often have low IoU due to rapid camera or object motion.

Multi-Domain Benchmark. Until recently, the MOTChallenge benchmark, focused on the multi-pedestrian tracking domain, was the primary benchmark for tracker evaluation. It enabled the verification of pivotal tracker methods [8, 89, 103]. Moreover, it was the only benchmark for *general* tracking. Trackers are often designed to specialize in a specific domain; for example, trackers [75, 76, 74, 61] are evaluated exclusively on MOTChallenge, while the tracker [43] is evaluated only on SportsMOT and SoccerNet [20]. Results on these benchmarks can sometimes be misleading. For instance, evaluating end-to-end methods only on MOTChallenge could lead to the incorrect conclusion that they generally underperform compared to tracking-by-detection methods. However, this is proven false when comparing

Method	Year	HOTA	AssA	DetA	MOTA	IDF1
<i>tracking-by-detection</i>						
ByteTrack [103]	2022	64.1	52.3	78.5	95.9	71.4
BoT-SORT [3]	2022	68.7	55.9	84.4	94.5	70.0
OC.SORT [15]	2023	73.7	61.5	<i>88.5</i>	96.5	74.0
MixSORT-OC [21]	2023	74.1	62.0	<i>88.5</i>	96.5	74.4
MixSORT-Byte [21]	2023	65.7	54.8	78.8	96.2	74.1
Deep-ElIoU [43]	2023	<i>77.2</i>	<i>67.7</i>	88.2	96.3	79.8
MotionTrack [91]	2024	74.0	61.7	88.8	96.6	74.0
ETTrack [2]	2024	74.3	62.1	88.8	96.8	74.5
MoveSORT [2]	2024	74.6	63.7	87.5	<i>96.7</i>	<i>76.9</i>
DeepMoveSORT [1]	2024	78.7	70.3	88.1	96.5	81.7
<i>end-to-end</i>						
MeMOTR [31]	2022	70.0	59.1	83.1	91.5	71.4
MOTIP [32]	2024	<u>71.9</u>	<u>62.0</u>	<u>83.4</u>	<u>92.9</u>	<u>75.0</u>

Table 6: Comparison of tracking methods on the SportsMOT benchmark. The best results for each metric are highlighted in bold, while the second-best results are highlighted in italics. The best results for each paradigm separately are underlined. The results for MoveSORT are taken from [2], for Deep-ElIoU and BoT-SORT are taken from [43], and all other results are taken from [21]

Method	Year	HOTA	AssA	DetA	MOTA	IDF1
<i>offline</i>						
SUSHI* [17]	2022	64.4	58.9	71.8	82.7	72.4
CoNo-Link* [33]	2024	<u>65.2</u>	<u>59.3</u>	<u>72.8</u>	<u>84.9</u>	<u>73.4</u>
<i>tracking-by-detection</i>						
ByteTrack [103]	2022	57.9	48.4	71.9	85.7	65.8
BoT-SORT [3]	2022	59.2	50.5	72.2	<i>85.6</i>	67.9
OC_SORT [15]	2023	59.0	50.5	71.6	83.1	65.9
SparseTrack [52]	2023	59.9	51.6	71.8	85.4	68.6
Deep OC_SORT [56]	2023	62.8	55.8	<u>72.4</u>	84.6	70.7
MoveSORT [2]	2024	58.7	49.4	72.2	84.4	65.3
DeepMoveSORT [1]	2024	62.4	55.2	72.1	84.4	70.4
UCMCTrack [△] [96]	2024	<u>63.9</u>	<u>58.1</u>	71.4	83.5	<u>72.1</u>
<i>end-to-end</i>						
MOTRv2 [104]	2024	67.4	62.2	73.6	<u>84.0</u>	75.8

Table 7: Comparison of tracking methods on MOT17, MOT20 and DanceTrack with weights macro-average metric values. The weights for MOT17, MOT20 and DanceTrack are 0.5, 0.5 and 1.0 respectively. The best results for each metric are highlighted in bold, while the second-best results are highlighted in italics. Methods marked with Δ employ test set information.

performance on DanceTrack.

To evaluate the general tracking performance of different methods, we collect and aggregate their metrics across multiple benchmarks. Since trackers are rarely evaluated on SportsMOT compared to MOTChallenge and DanceTrack, we construct two sets of aggregated results. The first set, presented in Table 7, includes weighted macro-averaged results with weights 0.5, 0.5, and 1.0 for MOT17, MOT20, and DanceTrack, respectively. Effectively, MOT17 and MOT20 are treated as a single domain. The second set, presented in Table 8, includes weighted macro-averaged results with weights 0.5, 0.5, 1.0, and 1.0 for MOT17, MOT20, DanceTrack, and SportsMOT, respectively.

Based on the results presented in Table 7, we can rank tracking paradigms by performance as follows: end-to-end, offline, and tracking-by-detection. The best-performing tracker is MOTRv2, with 67.4% HOTA and 75.8 IDF1. Compared to the best offline method, CoNo-Link, MOTRv2 achieves a 2.2% higher HOTA. Compared to the best online tracking-by-detection method, UCMCTrack, MOTRv2 outperforms it by 3.5% in terms of HOTA. Interestingly, ByteTrack performs best in terms of MOTA, achieving a score of 85.7%. ByteTrack outperforms MOTRv2 by 1.7% in MOTA. Two other methods with similar MOTA are BoT-SORT and SparseTrack, both of which closely resemble ByteTrack.

Unfortunately, results for *real* end-to-end methods on MOT20 are not available and are therefore missing from this benchmark. Since DETR tends to underperform on crowded images with small objects [16, 110], we infer poor performance, which is possibly the reason behind the missing results. MOTRv2 stands out, as it mitigates DETR’s limitations by using YOLOX for detection proposals. Lastly, we observe that offline methods based on GNNs outperform online variants within the tracking-by-detection paradigm.

When focusing solely on the online tracking-by-detection results in Table 7, UCMCTrack emerges as the best-performing tracker in scenarios where camera parameters can be estimated for ground plane projection. The second tier of trackers in terms of performance includes Deep OC_SORT and DeepMoveSORT, both of which employ a combination of improved motion models and appearance-based association. Other methods fall behind as they lack these critical components.

In Table 8, we observe only a few tracking-by-detection methods that publish results on all datasets. The best-performing method is DeepMoveSORT, with 67.9% in terms of HOTA and 74.1% in terms of IDF1. ByteTrack is the best tracker in terms of MOTA, achieving a score of 89.1%. Unfortunately, many trackers are missing as they lack evaluations on either MOT20 or SportsMOT. However, we present these results as baselines, hoping for more robust evaluations across multiple datasets in future MOT works. We note that a better alternative to this *improvisation* for general tracking evaluation would be a large-scale multi-domain MOT dataset.

Method	Year	HOTA	AssA	DetA	MOTA	IDF1
<i>tracking-by-detection</i>						
ByteTrack [103]	2022	60.0	58.5	65.4	89.1	67.6
BoT-SORT [3]	2022	62.4	61.8	66.8	<i>88.6</i>	68.6
OC-SORT [15]	2023	63.9	63.2	68.2	87.6	68.6
MoveSORT [2]	2024	64.0	<i>62.1</i>	<i>69.4</i>	88.5	<i>69.2</i>
DeepMoveSORT [1]	2024	67.9	60.2	77.4	88.4	74.1

Table 8: Comparison of tracking methods on MOT17, MOT20, DanceTrack, and SportsMOT with weights macro-average metric values. The weights for MOT17, MOT20, DanceTrack and SportsMOT are 0.5, 0.5, 1.0 and 1.0 respectfully. The best results for each metric are highlighted in bold, while the second-best results are highlighted in italics.

7. Conclusion

This paper presents a comprehensive survey with an in-depth analysis of deep learning-based methods in MOT, covering the field from its foundations to modern state-of-the-art approaches. We review and categorize tracking-by-detection methods into five categories: joint detection and embedding, heuristic-based, motion-based, affinity learning, and offline methods. Additionally, we review end-to-end MOT methods and compare them in detail with tracking-by-detection approaches. All recent methods, starting from the year 2022, are compared, and their results are analyzed on multiple datasets. Furthermore, the generality of these methods is assessed by evaluating their average performance across different domains.

We conclude that while heavily engineered heuristic-based methods achieve state-of-the-art performance on highly crowded datasets with linear object motion, such as MOTChallenge, methods that improve association using deep learning models prevail on datasets from other domains, such as DanceTrack and SportsMOT, demonstrating great potential. We primarily refer to methods based on affinity learning (including offline variants using GNNs) from the tracking-by-detection paradigm and all end-to-end methods, as they can learn to associate from data and require minimal hyperparameter tuning during inference. Finally, since many papers evaluate their methods on only one or two domains, we are unable to fully assess their generality. Future research would benefit from evaluating methods across multiple datasets to provide a more comprehensive understanding of generalization.

Acknowledgements

I would like to thank Professor Mirjana Maljković Ruzićić from the Faculty of Mathematics, University of Belgrade, for her helpful suggestions and guidance on the writing of this paper.

Appendix A. Kalman Filter

Kalman Filter. The Kalman filter (KF) is a Bayesian filter used to estimate the state of a dynamic system based on three key assumptions: (1) the process and observation models are linear, (2) the process noise—representing uncertainties in the process dynamics—and the measurement (observation) noise are Gaussian with zero mean and known covariance, and (3) the Markov assumption, which states that the future state depends only on the current state and not on past states. These assumptions allow the state estimate to evolve over time while preserving a Gaussian distribution throughout the process.

In practice, within the domain of multi-object tracking, these assumptions (e.g., known process and measurement models) are often relaxed and supplemented with heuristics and domain knowledge, while still achieving satisfactory results. In the following text, we briefly but formally explain the KF process.

We adopt a similar notation for the Kalman filter equations as in [69]. Let $\mathbf{z}_i$ represent the true state of a tracked object at time t_i . A motion model is used to compute the mean $\hat{\mathbf{z}}_{i+1}$ and covariance $\hat{\mathbf{P}}_{i+1}$ of the Gaussian prior as:

$$\hat{\mathbf{z}}_{i+1} = \mathbf{F}\hat{\mathbf{z}}_i \quad (\text{A.1})$$

$$\hat{\mathbf{P}}_{i+1} = \mathbf{F}\hat{\mathbf{P}}_i\mathbf{F}^\top + \mathbf{Q} \quad (\text{A.2})$$

Here, $\mathbf{F}$ is the state transition matrix, i.e., the motion model, $\mathbf{Q}$ represents the process noise covariance, and $\hat{\mathbf{z}}_i$, $\hat{\mathbf{P}}_i$ are posterior estimates from the previous time step. The measurement prior follows a Gaussian distribution with parameters:

$$\hat{\mathbf{x}}_{i+1} = \mathbf{H}\hat{\mathbf{z}}_{i+1} \quad (\text{A.3})$$

$$\hat{\mathbf{\Sigma}}_{i+1} = \mathbf{H}\hat{\mathbf{P}}_{i+1}\mathbf{H}^\top + \mathbf{R} \quad (\text{A.4})$$

After a new observation is received, the posterior distribution $\mathcal{N}(\tilde{\mathbf{z}}_{i+1}, \tilde{\mathbf{P}}_{i+1})$ is refined using:

$$\mathbf{K}_{i+1} = \hat{\mathbf{P}}_{i+1} \mathbf{H}^\top \hat{\Sigma}_{i+1}^{-1} \quad (\text{A.5})$$

$$\Delta \mathbf{x}_{i+1} = \mathbf{x}_{i+1} - \hat{\mathbf{x}}_{i+1} \quad (\text{A.6})$$

$$\tilde{\mathbf{z}}_{i+1} = \hat{\mathbf{z}}_{i+1} + \mathbf{K}_{i+1} \Delta \mathbf{x}_{i+1} \quad (\text{A.7})$$

$$\tilde{\mathbf{P}}_{i+1} = \hat{\mathbf{P}}_{i+1} - \mathbf{K}_{i+1} \hat{\Sigma}_{i+1} \mathbf{K}_{i+1}^\top \quad (\text{A.8})$$

where observations $\mathbf{x}_i$ are assumed to be samples from a normal distribution $\mathcal{N}(\mathbf{H}\mathbf{z}_i, \mathbf{R})$, in which case $\mathbf{H}$ maps states to observations, and $\mathbf{R}$ denotes the observation noise covariance matrix. The $\mathbf{K}_{i+1}$ is the *Kalman gain*, and $\Delta \mathbf{x}_{i+1}$ denotes the *innovation*. The posterior estimates provide the final state prediction at t_{i+1} .

In tracking-by-detection approaches, observations $\mathbf{x}$ capture noisy measurements of object’s position and size, i.e., bounding box outputs from object detection models. The KF prior facilitates data association between detections in consecutive frames, combining predictions with the associated detections via Bayesian inference. During this process, the KF smooths noisy detections, leveraging covariance matrices $\mathbf{Q}$ and $\mathbf{R}$, which are usually approximated using domain knowledge and heuristics.

The KF is a staple in tracking-by-detection algorithms [8, 89, 103, 3, 15, 56] due to its efficiency in modeling object dynamics and handling noisy observations, delivering satisfactory results on datasets where object motion is predominantly linear [8, 89, 103, 3, 15], such as MOT17 and MOT20 datasets [22]. However, this does not hold in the general case, where the performance of the KF declines compared to deep learning-based motion model alternatives [78, 91, 2, 1].

References

- [1] M. Adžemović, P. Tadić, A. Petrović, and M. Nikolić. Engineering an efficient object tracker for non-linear motion, 2024. URL <https://arxiv.org/abs/2407.00738>.
- [2] M. Adžemović, P. Tadić, A. Petrović, and M. Nikolić. Beyond kalman filters: deep learning-based filters for improved object tracking. *Machine Vision and Applications*, 36(1):20, 2024. ISSN 1432-1769. doi: 10.1007/s00138-024-01644-x. URL <https://doi.org/10.1007/s00138-024-01644-x>.
- [3] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. Bot-sort: Robust associations multi-pedestrian tracking. In *arXiv preprint*, 2022. Arxiv: 2206.14651.
- [4] I. Ahmed, S. Din, G. Jeon, F. Piccialli, and G. Fortino. "towards collaborative robotics in top view surveillance: A framework for multiple object tracking by detection using deep learning", "2021". ISSN "2329-9266". URL "<https://www.ieee-jas.net/en/article/doi/10.1109/JAS.2020.1003453>". "".
- [5] H. Ahn and H.-J. Cho. Research of multi-object detection and tracking using machine learning based on knowledge for video surveillance system. *Personal and Ubiquitous Computing*, 26(2): 385–394, 2022. ISSN 1617-4917. doi: 10.1007/s00779-019-01296-z. URL <https://doi.org/10.1007/s00779-019-01296-z>.
- [6] P. Bergmann, T. Meinhardt, and L. Leal-Taixe. Tracking without bells and whistles. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, page 941–951. IEEE, Oct. 2019. doi: 10.1109/iccv.2019.00103. URL <http://dx.doi.org/10.1109/ICCV.2019.00103>.
- [7] K. Bernardin, A. Elbs, and R. Stiefelhagen. Multiple object tracking performance metrics and evaluation in a smart room environment. *Proceedings of IEEE International Workshop on Visual Surveillance*, 01 2006.
- [8] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016. doi: 10.1109/ICIP.2016.7533003.
- [9] A. Bhowmik, P. Mettes, M. R. Oswald, and C. G. M. Snoek. Union-over-intersections: Object detection beyond winner-takes-all. In *13th International Conference on Learning Representations (ICLR) 2025*, 2025. URL <https://arxiv.org/abs/2311.18512>.
- [10] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao. Yolov4: Optimal speed and accuracy of object detection, 2020. URL <https://arxiv.org/abs/2004.10934>.
- [11] G. Bradski. The OpenCV Library. *Dr. Dobb’s Journal of Software Tools*, 2000.

- [12] G. Brasó and L. Leal-Taixé. Learning a neural solver for multiple object tracking. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- [13] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nuScenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11618–11628, Los Alamitos, CA, USA, June 2020. IEEE Computer Society. doi: 10.1109/CVPR42600.2020.01164. URL <https://doi.ieeecomputersociety.org/10.1109/CVPR42600.2020.01164>.
- [14] J. Cai, M. Xu, W. Li, Y. Xiong, W. Xia, Z. Tu, and S. Soatto. Memot: Multi-object tracking with memory. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8080–8090, 2022. doi: 10.1109/CVPR52688.2022.00792.
- [15] J. Cao, J. Pang, X. Weng, R. Khiroudkar, and K. Kitani. Observation-centric sort: Rethinking sort for robust multi-object tracking. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9686–9696, 2023. doi: 10.1109/CVPR52729.2023.00934.
- [16] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko. End-to-end object detection with transformers. In A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, editors, *Computer Vision – ECCV 2020*, pages 213–229, Cham, 2020. Springer International Publishing. ISBN 978-3-030-58452-8. doi: 10.1007/978-3-030-58452-8_13. URL https://doi.org/10.1007/978-3-030-58452-8_13.
- [17] O. Cetintas, G. Brasó, and L. Leal-Taixé. Unifying short and long-term tracking with graph hierarchies. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22877–22887, June 2023. doi: 10.1109/CVPR52729.2023.02191.
- [18] Q. Chen, X. Chen, J. Wang, S. Zhang, K. Yao, H. Feng, J. Han, E. Ding, G. Zeng, and J. Wang. Group detr: Fast detr training with group-wise one-to-many assignment. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2023.
- [19] G. Ciaparrone, F. Luque Sánchez, S. Tabik, L. Troiano, R. Tagliaferri, and F. Herrera. Deep learning in video multi-object tracking: A survey. *Neurocomputing*, 381:61–88, 2020. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2019.11.023>. URL <https://www.sciencedirect.com/science/article/pii/S0925231219315966>.
- [20] A. Cioppa, S. Giancola, A. Deliege, L. Kang, X. Zhou, Z. Cheng, B. Ghanem, and M. Van Droogenbroeck. Soccernet-tracking: Multiple object tracking dataset and benchmark in soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3491–3502, 2022.
- [21] Y. Cui, C. Zeng, X. Zhao, Y. Yang, G. Wu, and L. Wang. Sportsmot: A large multi-object tracking dataset in multiple sports scenes. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9887–9897, 2023. doi: 10.1109/ICCV51070.2023.00910.
- [22] P. Dendorfer, A. Ošep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, and L. Leal-Taixé. Motchallenge: A benchmark for single-camera multiple target tracking. *International Journal of Computer Vision*, 129(4):845–881, 2021. ISSN 1573-1405. doi: 10.1007/s11263-020-01393-0. URL <https://doi.org/10.1007/s11263-020-01393-0>.
- [23] X. Ding, Y. Guo, G. Ding, and J. Han. Acnet: Strengthening the kernel skeletons for powerful cnn via asymmetric convolution blocks. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1911–1920, 2019. doi: 10.1109/ICCV.2019.00200.
- [24] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=YicbFdNTTy>.
- [25] Y. Du, J. Wan, Y. Zhao, B. Zhang, Z. Tong, and J. Dong. Giauotracker: A comprehensive framework for mcmot with global information and optimizing strategies in visdrone 2021. In *2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 2809–2819, 2021. doi: 10.1109/ICCVW54120.2021.00315.

- [26] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, and H. Meng. Strongsort: Make deepsort great again. volume 25, pages 8725–8737, 2023. doi: 10.1109/TMM.2023.3240881.
- [27] O. Elharrouss, N. Almaadeed, and S. Al-Maadeed. A review of video surveillance systems. *Journal of Visual Communication and Image Representation*, 77:103116, 2021. ISSN 1047-3203. doi: <https://doi.org/10.1016/j.jvcir.2021.103116>. URL <https://www.sciencedirect.com/science/article/pii/S1047320321000729>.
- [28] P. Emami, P. M. Pardalos, L. Elefteriadou, and S. Ranka. Machine learning methods for data association in multi-object tracking. *ACM Comput. Surv.*, 53(4), Aug. 2020. ISSN 0360-0300. doi: 10.1145/3394659. URL <https://doi.org/10.1145/3394659>.
- [29] G. D. Evangelidis and E. Z. Psarakis. Parametric image alignment using enhanced correlation coefficient maximization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(10): 1858–1865, Oct 2008. ISSN 0162-8828. doi: 10.1109/TPAMI.2008.113.
- [30] T. Fischer, T. E. Huang, J. Pang, L. Qiu, H. Chen, T. Darrell, and F. Yu. Qdtrack: Quasi-dense similarity learning for appearance-only multiple object tracking. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(12):15380–15393, 2023. doi: 10.1109/TPAMI.2023.3301975.
- [31] R. Gao and L. Wang. Memotr: Long-term memory-augmented transformer for multi-object tracking. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9867–9876, 2023. doi: 10.1109/ICCV51070.2023.00908.
- [32] R. Gao, Y. Zhang, and L. Wang. Multiple object tracking as id prediction, 2024. URL <https://arxiv.org/abs/2403.16848>.
- [33] Y. Gao, H. Xu, N. Wang, J. Li, and X. Gao. Multi-scene generalized trajectory global graph solver with composite nodes for multiple object tracking, 2023. URL <https://arxiv.org/abs/2312.08951>.
- [34] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun. Yolox: Exceeding yolo series in 2021, 2021. URL <https://arxiv.org/abs/2107.08430>.
- [35] R. Girshick. Fast R-CNN. In *2015 IEEE International Conference on Computer Vision (ICCV)*, pages 1440–1448, Santiago, Chile, 2015. doi: 10.1109/ICCV.2015.169. URL <https://doi.org/10.1109/ICCV.2015.169>.
- [36] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 580–587, 2014. doi: 10.1109/CVPR.2014.81.
- [37] R. Girshick, I. Radosavovic, G. Gkioxari, P. Dollár, and K. He. Detectron. <https://github.com/facebookresearch/detectron>, 2018.
- [38] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. 2016. <http://www.deeplearningbook.org>.
- [39] X. Han, N. Oishi, Y. Tian, E. Ucurum, R. Young, C. Chatwin, and P. Birch. Ettrack: enhanced temporal motion predictor for multi-object tracking. *Applied Intelligence*, 55(1):33, 2024. ISSN 1573-7497. doi: 10.1007/s10489-024-05866-4. URL <https://doi.org/10.1007/s10489-024-05866-4>.
- [40] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- [41] K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask r-cnn, 2018. URL <https://arxiv.org/abs/1703.06870>.
- [42] A. Hossam, A. Ramadan, M. Magdy, R. Abdelwahab, S. Ashraf, and Z. Mohamed. Revolutionizing retail analytics: Advancing inventory and customer insight with ai, 2024.
- [43] H.-W. Huang, C.-Y. Yang, J. Sun, P.-K. Kim, K.-J. Kim, K. Lee, C.-I. Huang, and J.-N. Hwang. Iterative Scale-Up ExpansionIoU and Deep Features Association for Multi-Object Tracking in Sports . In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 163–172, 2024. doi: 10.1109/WACVW60836.2024.00024.

- [44] M. Hussain. Yolov5, yolov8 and yolov10: The go-to detectors for real-time vision, 2024. URL <https://arxiv.org/abs/2407.02988>.
- [45] D. M. Jiménez-Bravo, Álvaro Lozano Murciego, A. Sales Mendes, H. Sánchez San Blás, and J. Bajo. Multi-object tracking in traffic environments: A systematic literature review. *Neurocomputing*, 494:43–55, 2022. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2022.04.087>. URL <https://www.sciencedirect.com/science/article/pii/S0925231222004672>.
- [46] G. Jocher, J. Qiu, and A. Chaurasia. Ultralytics yolo, 2023. URL <https://ultralytics.com>. Available at <https://github.com/ultralytics/ultralytics>.
- [47] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=SJU4ayYgl>.
- [48] C. Lea, R. Vidal, A. Reiter, and G. D. Hager. Temporal convolutional networks: A unified approach to action segmentation. In G. Hua and H. Jégou, editors, *Computer Vision – ECCV 2016 Workshops*, pages 47–54, Cham, 2016. Springer International Publishing. ISBN 978-3-319-49409-8. doi: 10.1007/978-3-319-49409-8_7. URL https://doi.org/10.1007/978-3-319-49409-8_7.
- [49] Y. Liao, J. Xie, and A. Geiger. Kitti-360: A novel dataset and benchmarks for urban scene understanding in 2d and 3d. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(3):3292–3310, 2023. doi: 10.1109/TPAMI.2022.3179507.
- [50] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 936–944, 2017. doi: 10.1109/CVPR.2017.106.
- [51] S. Liu, F. Li, H. Zhang, X. Yang, X. Qi, H. Su, J. Zhu, and L. Zhang. DAB-DETR: Dynamic anchor boxes are better queries for DETR. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=oMI9Pj0b9Jl>.
- [52] Z. Liu, X. Wang, C. Wang, W. Liu, and X. Bai. Sparsetrack: Multi-object tracking by performing scene decomposition based on pseudo-depth. 2025. doi: 10.1109/TCSVT.2024.3524670.
- [53] J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe. Hota: A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2): 548–578, 2021. ISSN 1573-1405. doi: 10.1007/s11263-020-01375-2.
- [54] H. Luo, Y. Gu, X. Liao, S. Lai, and W. Jiang. Bag of tricks and a strong baseline for deep person re-identification. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1487–1495, 2019. doi: 10.1109/CVPRW.2019.00190.
- [55] W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu, and T.-K. Kim. Multiple object tracking: A literature review. *Artificial Intelligence*, 293:103448, 2021. ISSN 0004-3702. doi: <https://doi.org/10.1016/j.artint.2020.103448>. URL <https://www.sciencedirect.com/science/article/pii/S0004370220301958>.
- [56] G. Maggolino, A. Ahmad, J. Cao, and K. Kitani. Deep oc-sort: Multi-pedestrian tracking by adaptive re-identification. In *2023 IEEE International Conference on Image Processing (ICIP)*, pages 3025–3029, 2023. doi: 10.1109/ICIP49359.2023.10222576.
- [57] T. Meinhardt, A. Kirillov, L. Leal-Taixé, and C. Feichtenhofer. Trackformer: Multi-object tracking with transformers. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8834–8844, 2022. doi: 10.1109/CVPR52688.2022.00864.
- [58] M. Miah, G.-A. Bilodeau, and N. Saunier. Learning data association for multi-object tracking using only coordinates. *Pattern Recognition*, 160:111169, 2025. ISSN 0031-3203. doi: <https://doi.org/10.1016/j.patcog.2024.111169>. URL <https://www.sciencedirect.com/science/article/pii/S0031320324009208>.
- [59] K. O’Shea and R. Nash. An introduction to convolutional neural networks, 2015. URL <https://arxiv.org/abs/1511.08458>.

- [60] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32:8024–8035, 2019.
- [61] Z. Qin, S. Zhou, L. Wang, J. Duan, G. Hua, and W. Tang. Motiontrack: Learning robust short-term and long-term motions for multi-object tracking. In *CVPR*, 2023. doi: 10.1109/CVPR52729.2023.01720.
- [62] L. Rakai, H. Song, S. Sun, W. Zhang, and Y. Yang. Data association in multiple object tracking: A survey of recent techniques. *Expert Systems with Applications*, 192:116300, 2022. ISSN 0957-4174. doi: <https://doi.org/10.1016/j.eswa.2021.116300>. URL <https://www.sciencedirect.com/science/article/pii/S0957417421016031>.
- [63] L. Ramshaw, R. E. and Tarjan. On minimum-cost assignments in unbalanced bipartite graphs. In *HP Laboratories*, 2012.
- [64] R. Ravindran, M. J. Santora, and M. M. Jamali. Multi-object detection and tracking, based on dnn, for autonomous vehicles: A review. *IEEE Sensors Journal*, 21(5):5668–5677, 2021. doi: 10.1109/JSEN.2020.3041615.
- [65] J. Redmon and A. Farhadi. Yolo9000: Better, faster, stronger, 2016. URL <https://arxiv.org/abs/1612.08242>.
- [66] J. Redmon and A. Farhadi. Yolov3: An incremental improvement, 2018. URL <https://arxiv.org/abs/1804.02767>.
- [67] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016. doi: 10.1109/CVPR.2016.91.
- [68] S. Ren, K. He, R. Girshick, and J. Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *NIPS*, 2015. Arxiv: 1506.01497.
- [69] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. G. van Sloun, and Y. C. Eldar. Kalmannet: Neural network aided kalman filtering for partially known dynamics. *IEEE Transactions on Signal Processing*, 70:1532–1547, 2022. doi: 10.1109/TSP.2022.3158588.
- [70] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese. Generalized intersection over union. June 2019.
- [71] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi. Performance measures and a data set for multi-target, multi-camera tracking. In G. Hua and H. Jégou, editors, *Computer Vision – ECCV 2016 Workshops*, pages 17–35, Cham, 2016. Springer International Publishing. ISBN 978-3-319-48881-3. doi: 10.1007/978-3-319-48881-3_2. URL https://doi.org/10.1007/978-3-319-48881-3_2.
- [72] S. Shao, Z. Zhao, B. Li, T. Xiao, G. Yu, X. Zhang, and J. Sun. Crowddhuman: A benchmark for detecting human in a crowd. *arXiv preprint arXiv:1805.00123*, 2018.
- [73] M. Sohan, T. Sai Ram, and C. V. Rami Reddy. A review on yolov8 and its advancements. In I. J. Jacob, S. Piramuthu, and P. Falkowski-Gilski, editors, *Data Intelligence and Cognitive Informatics*, pages 529–545, Singapore, 2024. Springer Nature Singapore.
- [74] D. Stadler and J. Beyerer. An improved association pipeline for multi-person tracking. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 3170–3179, 2023. doi: 10.1109/CVPRW59228.2023.00319.
- [75] V. D. Stanojevic and B. T. Todorovic. Boosttrack: boosting the similarity measure and detection confidence for improved multiple object tracking. *Machine Vision and Applications*, 35(3):53, April 2024. ISSN 1432-1769. doi: 10.1007/s00138-024-01531-5. URL <https://doi.org/10.1007/s00138-024-01531-5>.
- [76] V. Stanojević and B. Todorović. Boosttrack++: using tracklet information to detect more objects in multiple object tracking, 2024. URL <https://arxiv.org/abs/2408.13003>.
- [77] P. Sun, J. Cao, Y. Jiang, R. Zhang, E. Xie, Z. Yuan, C. Wang, and P. Luo. Transtrack: Multiple object tracking with transformer, 2021. URL <https://arxiv.org/abs/2012.15460>.

- [78] P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani, and P. Luo. Dancetrack: Multi-object tracking in uniform appearance and diverse motion. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20961–20970, 2022. doi: 10.1109/CVPR52688.2022.02032.
- [79] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González. A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas. *Machine Learning and Knowledge Extraction*, 5(4):1680–1716, Nov. 2023. ISSN 2504-4990. doi: 10.3390/make5040083. URL <http://dx.doi.org/10.3390/make5040083>.
- [80] D. Touska, K. Gkountakos, T. Tsikrika, K. Ioannidis, S. Vrochidis, and I. Kompatsiaris. Graph-based data association in multiple object tracking: A survey. In D.-T. Dang-Nguyen, C. Gurrin, M. Larson, A. F. Smeaton, S. Rudinac, M.-S. Dao, C. Trattner, and P. Chen, editors, *MultiMedia Modeling*, pages 386–398, Cham, 2023. Springer Nature Switzerland.
- [81] J. R. R. Uijlings, K. E. A. van de Sande, T. Gevers, and A. W. M. Smeulders. Selective search for object recognition, 2013. URL <https://ivi.fnwi.uva.nl/isis/publications/2013/UijlingsIJCV2013>.
- [82] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS)*, NIPS’17, pages 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. URL https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- [83] A. Wang, H. Chen, L. Liu, K. CHEN, Z. Lin, J. Han, and G. Ding. YOLOv10: Real-time end-to-end object detection. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=tz83Nyb711>.
- [84] C.-Y. Wang, H.-Y. M. Liao, I.-H. Yeh, Y.-H. Wu, P.-Y. Chen, and J.-W. Hsieh. Cspnet: A new backbone that can enhance learning capability of cnn, 2019. URL <https://arxiv.org/abs/1911.11929>.
- [85] G. Wang, M. Song, and J.-N. Hwang. Recent advances in embedding methods for multi-object tracking: A survey, 2024. URL <https://arxiv.org/abs/2205.10766>.
- [86] Y.-H. Wang, J.-W. Hsieh, P.-Y. Chen, M.-C. Chang, H. H. So, and X. Li. Smiletrack: Similarity learning for occlusion-aware multiple object tracking, 2024. URL <https://arxiv.org/abs/2211.08824>.
- [87] Z. Wang, L. Zheng, Y. Liu, Y. Li, and S. Wang. Towards real-time multi-object tracking. In A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, editors, *Computer Vision – ECCV 2020*, pages 107–122, Cham, 2020. Springer International Publishing. ISBN 978-3-030-58621-8. doi: 10.1007/978-3-030-58621-8_7. URL https://doi.org/10.1007/978-3-030-58621-8_7.
- [88] C. Williams and C. Rasmussen. Gaussian processes for regression. In D. Touretzky, M. Mozer, and M. Hasselmo, editors, *Advances in Neural Information Processing Systems*, volume 8. MIT Press, 1995. URL https://proceedings.neurips.cc/paper_files/paper/1995/file/7cce53cf90577442771720a370c3c723-Paper.pdf.
- [89] N. Wojke, A. Bewley, and D. Paulus. Simple online and realtime tracking with a deep association metric. In *2017 IEEE International Conference on Image Processing (ICIP)*, pages 3645–3649, 2017. doi: 10.1109/ICIP.2017.8296962.
- [90] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick. Detectron2. <https://github.com/facebookresearch/detectron2>, 2019.
- [91] C. Xiao, Q. Cao, Y. Zhong, L. Lan, X. Zhang, Z. Luo, and D. Tao. Motiontrack: Learning motion predictor for multiple object tracking. *Neural Networks*, 179:106539, 2024. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2024.106539>.
- [92] Z. Xu, X. Zhan, Y. Xiu, C. Suzuki, and K. Shimada. Onboard dynamic-object detection and tracking for autonomous robot navigation with rgb-d camera. In *IEEE Robotics and Automation Letters*, 2023. Arxiv: 2303.00132.

- [93] F. Yang, S. Odashima, S. Masui, and S. Jiang. Hard to track objects with irregular motions and similar appearances? make it easier by buffering the matching space. In *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 4788–4797, 2023. doi: 10.1109/WACV56688.2023.00478.
- [94] M. Yang, G. Han, B. Yan, W. Zhang, J. Qi, H. Lu, and D. Wang. Hybrid-sort: Weak cues matter for online multi-object tracking. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 6504–6512, 2024. doi: 10.1609/aaai.v38i7.28471.
- [95] M. Yasir, S. Liu, S. Pirasteh, M. Xu, H. Sheng, J. Wan, F. A. de Figueiredo, F. J. Aguilar, and J. Li. Yoloshiptracker: Tracking ships in sar images using lightweight yolov8. *International Journal of Applied Earth Observation and Geoinformation*, 134:104137, 2024. ISSN 1569-8432. doi: <https://doi.org/10.1016/j.jag.2024.104137>. URL <https://www.sciencedirect.com/science/article/pii/S1569843224004916>.
- [96] K. Yi, K. Luo, X. Luo, J. Huang, H. Wu, R. Hu, and W. Hao. Ucmctrack: Multi-object tracking with uniform camera motion compensation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(7):6702–6710, Mar. 2024. doi: 10.1609/aaai.v38i7.28493.
- [97] E. Yu, T. Wang, Z. Li, Y. Zhang, X. Zhang, and W. Tao. MOTRv3: Release-fetch supervision for end-to-end multi-object tracking, 2024. URL <https://openreview.net/forum?id=ezPbPoYFME>.
- [98] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell. Bdd100k: A diverse driving dataset for heterogeneous multitask learning. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2633–2642, 2020. doi: 10.1109/CVPR42600.2020.00271.
- [99] F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang, and Y. Wei. Motr: End-to-end multiple-object tracking with transformer. In *European Conference on Computer Vision (ECCV)*, 2022.
- [100] H. Zhang, F. Li, S. Liu, L. Zhang, H. Su, J. Zhu, L. Ni, and H.-Y. Shum. DINO: DETR with improved denoising anchor boxes for end-to-end object detection. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=3mRwyG5one>.
- [101] L. Zhang, J. Gao, Z. Xiao, and H. Fan. Animaltrack: A benchmark for multi-animal tracking in the wild. *Int. J. Comput. Vision*, 131(2):496–513, Nov. 2022. ISSN 0920-5691. doi: 10.1007/s11263-022-01711-8. URL <https://doi.org/10.1007/s11263-022-01711-8>.
- [102] Y. Zhang, C. Wang, X. Wang, W. Zeng, and W. Liu. Fairmot: On the fairness of detection and re-identification in multiple object tracking. *International Journal of Computer Vision*, 129(11):3069–3087, Sept. 2021. ISSN 1573-1405. doi: 10.1007/s11263-021-01513-4. URL <http://dx.doi.org/10.1007/s11263-021-01513-4>.
- [103] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. Bytetrack: Multi-object tracking by associating every detection box. In S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, editors, *Computer Vision – ECCV 2022*, pages 1–21, Cham, 2022. Springer Nature Switzerland. ISBN 978-3-031-20047-2. doi: 10.1007/978-3-031-20047-2_1.
- [104] Y. Zhang, T. Wang, and X. Zhang. Motrv2: Bootstrapping end-to-end multi-object tracking by pretrained object detectors. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22056–22065, 2023. doi: 10.1109/CVPR52729.2023.02112.
- [105] Y. Zhang, X. Wang, X. Ye, W. Zhang, J. Lu, X. Tan, E. Ding, P. Sun, and J. Wang. Bytetrackv2: 2d and 3d multi-object tracking by associating every detection box. In *Combined Ophthalmic Research Rotterdam*, 2023. Arxiv: 2303.15334.
- [106] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen. Detrs beat yolos on real-time object detection. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16965–16974, 2024. doi: 10.1109/CVPR52733.2024.01605.
- [107] L. Zheng, Z. Bie, Y. Sun, J. Wang, C. Su, S. Wang, and Q. Tian. MARS: A video benchmark for large-scale person re-identification. In B. Leibe, J. Matas, N. Sebe, and M. Welling, editors, *Computer Vision – ECCV 2016*, pages 868–884, Cham, 2016. Springer International Publishing. ISBN 978-3-319-46466-4. doi: 10.1007/978-3-319-46466-4_52. URL https://doi.org/10.1007/978-3-319-46466-4_52.

- [108] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020. ISSN 2666-6510. doi: 10.1016/j.aiopen.2021.01.001. URL <https://www.sciencedirect.com/science/article/pii/S2666651021000012>.
- [109] X. Zhou, D. Wang, and P. Krähenbühl. Objects as points. In *arXiv preprint arXiv:1904.07850*, 2019.
- [110] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai. Deformable {detr}: Deformable transformers for end-to-end object detection. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=gZ9hCDWe6ke>.